Resource-Constrained Project Scheduling with Time-of-Use Energy Tariffs and Machine States: A Logic-Based Benders Decomposition approach

Corentin Juvigny[a], Antonín Novák[a], Jan Mandík[a], Zdeněk Hanzálek[a]

[a] *Czech Institute of Informatics, Robotics and Cybernetics, Czech Technical University in Prague, Jugoslávských partyzánů 1580/3, 160 00, Prague 6, Czech Republic*

arXiv:2601.06542v2 [math.OC] 6 Jul 2026

Abstract

In this paper, we investigate the *Resource-Constrained Project Scheduling Problem* (RCPSP) with *Time-of-Use* (TOU) *energy tariffs* and *machine states*, a variant of RCPSP for production scheduling, where energy price is part of the criteria and one highly energy-demanding machine can be in one of the following three states: `proc`, `idle`, or `off`. The problem involves scheduling all tasks, respecting precedence constraints and resource limitations, while minimizing the combination of the overall makespan and the Total Energy Cost (TEC), which varies according to the TOU tariffs, which can take negative values. We propose two novel approaches to solve it: a *monolithic Constraint Programming* (CP) approach and a *Logic-Based Benders Decomposition* (LBBD) approach. The latter combines a master problem handling the energy cost solved using *Integer Linear Programming* (ILP) with a subproblem handling the RCPSP, resolved using CP. Both approaches outperform the *monolithic compact* ILP counterpart, but the LBBD significantly outperforms the monolithic CP in most cases, especially when the makespan criterion is not included in the objective function, solving to optimality instances with up to 480 tasks. Finally, we propose a way to generalize our LBBD approach to other problems sharing similar characteristics, and applied it to various problems, such as an RCPSP with blocking times & total weighted tardiness criterion, or a flexible job shop.

Keywords: Resource-Constrained Project Scheduling Problems, Energy-price-aware Scheduling, Logic-Based Benders Decomposition, Mixed-Integer Linear Programming, Constraint Programming

1. Introduction

The industrial sector is a very energy-demanding area, accounting for around half of the total energy consumption of the world, a share that has never stopped growing in recent years (Fang et al., 2011). The rising costs induced by energy production, combined with increasing concerns about the viability of intensified production as well as new public efforts fostering more durable production processes, have led an increasing number of studies to concentrate on energy-aware project scheduling (Du et al., 2021).

Time-of-use (TOU) tariffs have emerged as a widely adopted strategy among energy suppliers of industrialized nations. This dynamic pricing mechanism serves two primary purposes: it facilitates the balancing of daily energy consumption patterns and mitigates peak load concentrations during specific hours (Hung and Michailidis, 2018; Chen and Zhang, 2019). It consists of adjusting the electricity tariffs according to the fluctuation of demand, resulting in variable tariffs that can change quickly. This pricing structure provides consumers with significant opportunities to reduce the costs of their energy use by shifting their major consumption activities to off-peak periods (Geng et al., 2020).

In industry, some energy-intensive operations, such as furnaces or industrial ovens, may benefit from a more efficient management strategy that involves maintaining a machine in an intermediate state during periods of inactivity, rather than completely shutting it down. Indeed, turning on a machine may consume a significant

Email addresses: `corentin.juvigny@cvut.cz` (Corentin Juvigny), `antonin.novak@cvut.cz` (Antonín Novák), `zdenek.hanzalek@cvut.cz` (Zdeněk Hanzálek)

amount of energy and time. However, the addition of these machine states leads to complex optimization problems (Benedikt et al., 2026). In real industrial processes, not all operations are energy-intensive. They may be part of a more global project scheduling, with precedence constraints between the processes and limited capacities on resources involved, a category of scheduling problems called *Resource-Constrained Project Scheduling Problems* (RCPSPs) (Herroelen et al., 1998).

To address these challenges, we study an RCPSP problem with TOU tariffs and machine states. The states of energy intensive resource are modeled in a transition diagram, indicating the duration of the transitions between states and their associated energy consumptions. We assume that this energy-intensive resource has a unitary capacity. The other resources have integer capacities. Moreover, a task requiring this energy-intensive resource does not use other resources. We consider two objective functions. The first minimizes the total energy cost (TEC) induced by energy-intensive tasks with respect to TOU tariffs, and can be denoted as $PS, 1\text{TOU}|\text{prec,states}|\text{TEC}$ in the extended Graham's three-field notation (Graham et al., 1979). The second combines the previous TEC with a minimization of the makespan over all tasks, i.e., $PS, 1\text{TOU}|\text{prec,states}|\text{TEC}, C_{\max}$. The reader can refer to Table A.1 of the supplementary materials for a summary of the acronyms used throughout this paper.

1.1. Paper contributions and outline

The main contributions of this article are the following.

- We introduce a new problem combining the RCPSP with machine states and TOU tariffs. To model it, we detail two compact models, which we henceforth refer to as *monolithic* models, one based on *Constraint Programming* (CP) and one based on *Integer Linear Programming* (ILP). We say monolithic because they model the problem as a whole and are not decomposed.
- We put forth a *Logic-based Benders decomposition* (LBBD) approach to solve the problem more efficiently. Experiments outline to what extent it outperforms monolithic formulations, especially in larger instances or when the makespan criterion is not considered, i.e., for $PS, 1\text{TOU}|\text{prec,states}|\text{TEC}$.
- We demonstrate the *generality* of our LBBD approach, which segregates energy-related parameters within the master problem and employs a classical scheduling model as its subproblem. In particular, we apply the LBBD approach to problems based on an RCPSP with blocking times & total weighted tardiness criterion ($PSm, 1\text{TOU}|\text{intree,states}|\text{TEC}, \sum_j w_j T_j$), and a flexible job shop ($FJm, 1\text{TOU}|\text{prec,states}|\text{TEC}, C_{\max}$).

The article is organized as follows. In Section 2, we review the current state of the art literature and delineate the position of our article among them. In Section 3, we present our hybrid energy-aware / RCPSP variant and propose an MILP formulation to solve it. Then, we present a CP approach in Section 4 and a new LBBD-based approach in Section 5. We assess and compare the proposed approaches in Section 6. Afterwards, Section 7 investigates the use of our LBBD approach to other problems. Finally, Section 8 concludes the article and exposes future avenues of research.

2. Literature Review

This section surveys the three research streams that converge in the present work: resource-constrained project scheduling, energy-aware scheduling under time-of-use tariffs with machine-states, and logic-based Benders decomposition for scheduling. For each stream, we identify the modeling choices and methodological limits that motivate our contribution, before making the relationship explicit at the end of the section.

2.1. Resource-Constrained Project Scheduling

The Resource-Constrained Project Scheduling Problems (RCPSPs), introduced by Wiest (1967), are widely studied strongly NP-Hard problems (Blazewicz et al., 1983), in which solving even moderate instances to optimality remains a computational challenge. The literature has explored the problem along two methodological axes. *Exact methods* include branch-and-bound (Möhring et al., 2003), constraint programming (Berthold et al.,

Reference	Machine env.	TOU	States	Objective	Method
Maghsoudlou et al. (2021)	RCPSP (multi-skill)	✓	–	TEC	Metaheuristic
Pouramin et al. (2024)	RCPSP (multi-mode)	✓	–	$C_{\max}$, cost	ILP/Metaheuristic
Gaggero et al. (2023)	Parallel machines	✓	–	$C_{\max}$, TEC	MILP/heuristic
Xiechen et al. (2026)	Unrel. parallel mach. (stoch.)	✓	–	$C_{\max}$, TEC	MILP/MISOCP, approx.
Zhao et al. (2025)	Job shop	✓	–	TEC	Metaheuristic
Ho et al. (2022)	Flow shop	✓	–	TEC	MILP
Shrouf et al. (2014)	1 machine	✓	✓	TEC	Genetic algorithm
Aghelinejad et al. (2018)	1 machine	✓	✓	TEC	ILP
Aghelinejad et al. (2019)	1 machine	✓	✓	TEC	Complexity results
Benedikt et al. (2020)	1 machine	✓	✓	TEC	ILP/CP + SPACES
Benedikt et al. (2026)	1 machine	✓	✓	TEC	Branch and Bound
Hall and Posner (2026)	1 machine (var. speeds)	✓	✓	TEC	Strongly poly. alg.
Zuccato et al. (2025)	Flexible Job Shop	–	✓	$C_{\max}$, TEC, $\sum T_j$	CP
Naderi and Roshanaei (2022)	Flexible Job Shop	–	–	$C_{\max}$	LBBD
This work	**RCPSP**	✓	✓	$\boldsymbol{C_{\max}}$**, TEC**	**LBBD**

Table 1: Comparison of the most closely related works. *States* refers to explicit power-state transition graphs on the energy-intense resource; *TOU* to time-of-use energy tariffs; *MISOCP* to mixed-integer second-order cone program.

2010; Laborie et al., 2018), and hybrid MIP/CP approaches (Chakrabortty et al., 2015); constraint programming has proved especially competitive (Heinz et al., 2025) for makespan criterion. *Heuristic methods* encompass priority-rule dispatching and metaheuristics (Munlin, 2018; Pellerin et al., 2020). A comprehensive survey on this topic has recently been published by Artigues et al. (2025).

However, none of the RCPSP variants surveyed above model the internal power dynamics of the constrained resources. Machine dynamics are often disregarded, and energy consumption, when accounted for, is uniformly attributed to processing modes (Zheng and Wang, 2015; Maghsoudlou et al., 2021; Pouramin et al., 2024). This is precisely the modeling gap we aim to fill by coupling RCPSP with explicit machine-state transitions on the energy-intensive resource.

2.2. Energy-Aware Scheduling Under TOU Tariffs and Machine States

TOU scheduling without machine states. A substantial body of work minimizes electricity costs under time-of-use (TOU) tariffs on classical machine environments: parallel machines (Ding et al., 2015; Gaggero et al., 2023), job shop (Kurniawan et al., 2021; Zhao et al., 2025), and flow shop and hybrid flow shop (Fazli Khalaf and Wang, 2018; Ho et al., 2022). Multi-objective extensions trade off energy cost against makespan (Schulz et al., 2020; Park and Ham, 2022; Gaggero et al., 2023) or tardiness (Rocholl et al., 2020; Kurniawan et al., 2021). More recently, Xiechen et al. (2026) extended this setting to unrelated parallel machines with uncertain processing times, proposing a chance-constrained bi-objective formulation that they convert into deterministic MILP and MISOCP models, together with a $(1+\epsilon)$-approximation scheme for the makespan–TEC Pareto front. A comprehensive review of this stream is provided by Gahm et al. (2016). Although special TOU structures can yield polynomial algorithms (Fang et al., 2016), the general case remains strongly NP-Hard, and all these formulations share the assumption that, in any given time period, a machine is either actively processing or completely shut down.

Single-machine TOU scheduling with power-state transitions. A tighter and more realistic model allows a machine to transition between a set of power states whose switch costs and durations interact non-trivially with the TOU tariff. Shrouf et al. (2014) introduced this setting for the single-machine problem $1, \text{TOU}|\text{states}|\text{TEC}$ and solved it with genetic algorithms. Aghelinejad et al. (2018) proposed a compact ILP formulation, and Aghelinejad et al. (2019) established that while fixing the job sequence yields a polynomial subproblem, the problem with free sequencing is strongly NP-Hard. The decisive methodological advance came with Benedikt et al. (2020), who observed that, given a fixed job sequence, the optimal state trajectory can be computed in polynomial time via a pre-processing step called SPACES which is based on shortest-path calculation in time-state graph. This reduces

the ILP/CP model size dramatically. Building on this, Benedikt et al. (2026) designed a dedicated branching scheme with tailored heuristics and achieved speedups of two orders of magnitude over Benedikt et al. (2020). In a parallel development, Hall and Posner (2026) extended the Shrouf et al. (2014) setting with q discrete processing speeds on top of *sleep* and *powered-down* idle states, and obtained the first *strongly polynomial-time* exact algorithm for this class via a block decomposition combined with a shortest-path argument, outperforming CPLEX by several orders of magnitude. They further showed that two seemingly minor changes—restricting speed changes to task boundaries, or allowing power-down at any task completion—render the problem strongly NP-complete, locating the single-machine variant sharply at the tractability frontier. These works establish a clear progression on the *single-machine* variant. What they leave open is whether the structural insight behind SPACES—the decoupling of scheduling decisions from state-trajectory optimization—survives when the machine is one of *several resources* competing for activities in an RCPSP, and how a decomposition should be organized to exploit it in that richer context.

RCPSP with energy and machine states. To the best of our knowledge, no prior work combines RCPSP with power-state machine decisions on the energy-intensive resources. Zuccato et al. (2025) recently incorporated machine modes, setup operations, and transport times into a flexible job-shop variant via constraint programming, but their model does not include explicit power-state transition graphs or TOU tariffs. The RCPSP papers that do address energy consumption (Zheng and Wang, 2015; Maghsoudlou et al., 2021; Pouramin et al., 2024) model energy through multi-modal activity durations rather than machine-state dynamics, and therefore do not capture the idle-period optimization that is central to the present problem.

2.3. Logic-Based Benders Decomposition for Scheduling

Classical Benders decomposition (Benders, 1962) reformulates a problem as a mixed-integer linear programming (MILP) master problem coupled with a continuous linear programming (LP) subproblem. The algorithm iteratively augments the master problem with *Benders cuts* derived from each integer-feasible solution of the master, and terminates once an optimal solution is obtained. However, this approach is inherently restricted to cases in which the subproblem admits a linear formulation, thereby limiting its applicability to a relatively narrow class of combinatorial optimization problems. To address this limitation, Hooker and Ottosson (2003) introduced *Logic-Based Benders Decomposition* (LBBD), which extends the framework to allow integer or more general combinatorial subproblems, with problem-specific inference procedures used to derive cutting planes from the subproblem solutions. The effectiveness of integrating MILP master problems with constraint programming (CP) subproblems was empirically demonstrated by Jain and Grossmann (2001) and Hooker (2007) in the context of scheduling, where substantial computational speedups were reported relative to purely MILP-based or CP-based methods.

More recently, LBBD has gained traction in energy- and sustainability-aware scheduling: Lei et al. (2022) applies it to parallel machine scheduling of step-deteriorating jobs minimizing a combination of tardiness and extra energy consumption, Hu et al. (2024) develops a bi-objective ε-LBBD for parallel machine selection with release dates and resource consumption, Yang et al. (2024) tackles order acceptance and scheduling on heterogeneous factories under carbon caps through a Branch-and-Check LBBD scheme, and Seman et al. (2024) adapts the framework to energy-aware task scheduling of nanosatellite constellations. These works confirm the suitability of LBBD whenever the energetic component naturally decouples into a global cost/capacity master problem and a per-resource sequencing subproblem.

2.4. Positioning the Present Contribution

The foregoing analysis delineates a specific research gap at the intersection of the three literature streams reviewed above: no existing study jointly addresses the RCPSP with TOU tariffs while simultaneously incorporating power-state machine decision modeling, and no available logic-based Benders decomposition (LBBD) framework proposes specialized cutting planes for this combined setting. Table 1 articulates this gap explicitly by contrasting the core characteristics of the most relevant prior contributions.

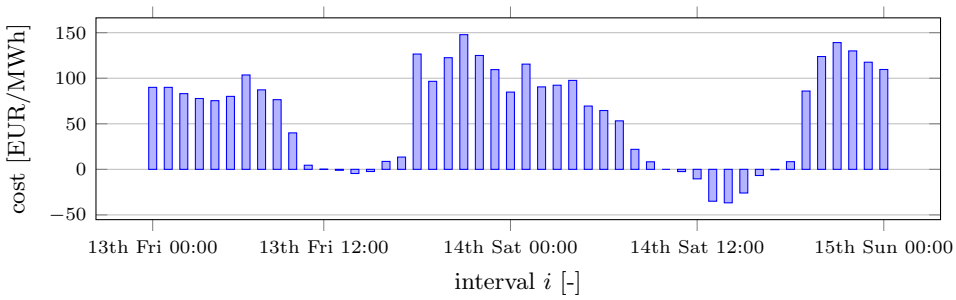


Figure 1: Example of a 48-hour part of the energy profile with real electricity costs in the day-ahead market in the Czech Republic from June 2025 provided by Czech OTE.

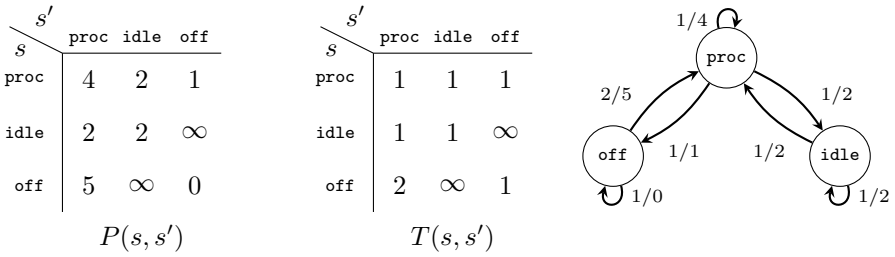


Figure 2: Parameters of the transition power function $P(s, s')$ and transition time function $T(s, s')$, and the corresponding transition graph, where every edge from s to s' is labeled by $T(s, s')/P(s, s')$.

The present paper extends previous work through three novel contributions, elaborated in the following sections. First, the new variant of the problem is a *strict generalization* of the single-machine problems studied by Benedikt et al. (2020) and Benedikt et al. (2026) including the RCPSP components in the energy problem. Second, the proposed LBBD exploits the SPACES shortest-path structure (Benedikt et al., 2020) as a natural decomposition boundary; its novelty lies in the tailored Benders cuts derived from the state-transition subproblem. Third, we show that the same architecture can be applied to many scheduling problem coupling combinatorial assignment with TOU-aware machine-state decisions, something that has not been done yet in the literature. In particular, we expose two examples, in which the assignment part is an RCPSP with blocking time periods and a flexible job-shop scheduling, respectively. This approach is interesting in real applications as it permits smooth integrations of the energy component to scheduling problems for which efficient solving algorithms exist.

3. Problem statement

In this section, we describe the RCPSP with machine states and time-of-use (TOU) tariffs, along with its modeling by a compact monolithic ILP. Consider a *time horizon* $h \in \mathbb{N}$ divided into non-overlapping *unitary intervals* $\mathcal{I} = \{1, \ldots, h\}$. Each interval is associated with an *energy cost* given by the vector $c = (c_1, c_2, \ldots, c_h) \in \mathbb{R}^h$. An example of a real energy cost profile is displayed in Figure 1.

Let $\mathcal{R} = \{R_0, R_1, \ldots, R_m\}$ be a set of *resources*, with $m \in \mathbb{N}$. We note ρ_k the *resource capacity* of a resource $k \in \mathcal{R}$. Let $\mathcal{T} = \{j_1, j_2, \ldots, j_n\}$ be a set of *non-preemptive tasks* to be processed. A *processing time* p_j and a *resource requirement* r_k^j, $k \in \mathcal{R}$ are associated to each task $j \in \mathcal{T}$. We want to calculate the *starting time* S_j of the task $j \in \mathcal{T}$, from which we can induce its *completion time* $C_j = S_j + p_j$. For each resource $k \in \mathcal{R}$, the sum of the resource requirements for the tasks processed simultaneously must not exceed the capacity of that resource. Moreover, all tasks must be processed before the end of the time horizon.

Additionally, let $G = (V, A)$ be the directed graph that represents the *precedence constraints* between the tasks. Each node of G is a task of $\mathcal{T}$, and each arc $(v_j, v_{j'}) \in A$ means that task j precedes task j', i.e.:

$$S_{j'} \geq C_j \iff S_{j'} - S_j \geq p_j, \quad \forall (j, j') \in A \tag{3.1}$$

We assume that the resource R_0 is the only *energy-intensive* resource, with a capacity equal to 1. We note $\mathcal{J} \subset \mathcal{T}$, the subset of *energy-intensive tasks*, i.e., such that $\forall j \in \mathcal{J}, r_0^j > 0$. We consider this subset to be non-empty. We note $\overline{\mathcal{J}}$ its *complementary*, i.e., $\overline{\mathcal{J}} = \mathcal{T} \setminus \mathcal{J}$.

Until then, the formulation resembles a classical RCPSP problem. Now, let us introduce the variables and constraints that are specific to our problem. We consider that the resource R_0 has a *three-state set* $\Sigma = \{\texttt{proc}, \texttt{idle}, \texttt{off}\}$. During each time interval, the machine operates in one of these states. When an energy-intensive task is processed, the machine must be in `proc` state. We define a *transition time* function $T : \Sigma \times \Sigma \to \mathbb{Z}_{>0} \cup \{\infty\}$ and a *transition energy* function $P : \Sigma \times \Sigma \to \mathbb{Z}_{\geq 0} \cup \{\infty\}$. We assume that the transition time between two states cannot be null. A transition time of value ∞ between two states denotes that this transition is infeasible. Benedikt et al. (2020) have demonstrated that it is possible to precompute the optimal transition with respect to the energy cost between any two `proc` intervals, as well as from the initial `off` state to any `proc` state, and from any `proc` state to the final `off` state; thanks to a graph-based method called *SPACES*. It consists of calculating the optimal transition between two (state, interval) as the shortest path between these states in the transition graph. An example of SPACES graph can be found in the supplementary materials. Let us denote by $\Omega = (\Omega_1, \Omega_2, \ldots, \Omega_h)$ the vector that indicates for each time interval the current transition of the machine. For example, $\Omega_t = (\texttt{off}, \texttt{proc})$ means that the machine is in transition from the `off` state at time t to the `proc` state at time $t+1$, while $\Omega_{t'} = (\texttt{idle}, \texttt{idle})$ signifies the machine stays in the `idle` state between times t' and $t'+1$. An optimal transition from one state in an interval t to another state in t' (assuming $t' > t$) in *SPACES* is therefore a vector containing successive transitions with the lowest transition costs. In addition, we assume that this machine starts and ends in the `off` state. Hence, we can define $\text{StartTime} = \{1, \ldots, T(\texttt{off}, \texttt{proc}) + 1\}$, $\text{EndTime}_j = \{h - T(\texttt{proc}, \texttt{off}) - p_j + 1, \ldots, h\}$ the sets during which no energy-intensive tasks can be processed. The sets and parameters are listed in Tables 2 and 3, respectively.

A *solution* of our problem is a pair (σ, Ω) where $\sigma = (\sigma_1, \sigma_2, \ldots, \sigma_n) \in \mathbb{Z}_{\geq 0}^n$ is the vector denoting the start time of each task and $\Omega = \{\Omega_1, \ldots, \Omega_h\} \in (\Sigma \times \Sigma)^h$ the vector of machine-state transitions. In summary, a solution is *feasible* if the following conditions are met:

- the resource capacities are not exceeded;
- the precedence constraints of each task are satisfied;
- the energy-demanding R_0 tasks are processed only when the R_0 machine is in `proc` state;
- the machine is in `off` state during the first and last intervals;
- the transition between states s and s' takes $T(s, s')$ time intervals.

The Total Energy Cost (TEC) of a solution (σ, Ω) can be computed as:

$$TEC = \sum_{t \in \mathcal{I}} c_t \cdot P(\Omega_t) \tag{3.2}$$

where for all $(s, s') = \Omega_t \in \Sigma^2,\ P(\Omega_t) = P(s, s')$.
We can compute the makespan $C_{\max}$ of a solution as:

$$C_{\max} = \max_{j \in \mathcal{T}}\ C_j \tag{3.3}$$

We aim to minimize the convex combination of both objectives. For that, we introduce a real parameter $\alpha \in [0, 1]$ that represents the balance between the two. In order to have better control over the balance between the two objective values, we normalize the individual objectives by their optimal solutions they would have if treated independently, i.e., the optimal makespan (lb_{RCPSP}) of the RCPSP problem for (3.3) and the optimal solution value for TEC (lb_{TEC}) when removing the non-energy-intensive tasks from the problem. However, their minimal impact on energy-intensive tasks is preserved, that is, we update their precedence constraints and their computation times so that, if the removed tasks were introduced in an optimal solution, but with an infinite resource capacity on all resources but R_0, they would fit between the energy-intensive tasks according to the precedence constraints. In case lb_{TEC} has a negative value (which in extreme cases may exist), we take

Sets	
$\mathcal{I} = \{1, \ldots, h\}$	Set of unitary time intervals over the horizon h
$\mathcal{R} = \{R_0, R_1, \ldots, R_m\}$	Set of resources, where R_0 is the energy-intensive resource
$\mathcal{T} = \{j_1, j_2, \ldots, j_n\}$	Set of non-preemptive tasks to be processed
$\mathcal{J} \subset \mathcal{T}$	Subset of energy-intensive tasks (i.e., $\forall j \in \mathcal{J}, r_0^j > 0$)
$\overline{\mathcal{J}} = \mathcal{T} \setminus \mathcal{J}$	Complementary subset of non-energy-intensive tasks
$\Sigma = \{\texttt{proc}, \texttt{idle}, \texttt{off}\}$	Set of states of the energy-intensive machine R_0
$G = (V, A)$	Directed precedence graph: nodes V are tasks of $\mathcal{T}$, arcs A encode precedences
StartTime = $\{1, \ldots, 1 + T(\texttt{off}, \texttt{proc})\}$	Set of intervals during which no energy-intensive task can start (machine warm-up from `off` to `proc`)
EndTime_j = $\{h - T(\texttt{proc}, \texttt{off}) - p_j + 1, \ldots, h\}$	Set of intervals during which energy-intensive task $j \in \mathcal{J}$ cannot be processed (machine shutdown from `proc` to `off`)

Table 2: Sets defined in the problem statement.

Parameters	
ρ_k	Capacity of resource $k \in \mathcal{R}$
r_k^j	Requirement on resource $k \in \mathcal{R}$ of task $j \in \mathcal{T}$
p_j	Processing time of task $j \in \mathcal{T}$
c_t	Energy cost at time $t \in \mathcal{I}$
c_{jt}^{job}	Cost of starting the job $j \in \mathcal{J}$ at interval $t \in \mathcal{I}$
c_{lm}^*	Cost of the optimal transition from interval $l \in \mathcal{I}$ to interval $m \in \mathcal{I}$ (`proc` to `proc`), except for intervals $l = 0$ and $m = h$ for which `off` state is considered instead of `proc`

Table 3: Parameters of the problem statement.

the absolute value of the objective so that lb_{TEC} is always a strictly positive value (if the optimal cost TEC is zero, we fix lb_{TEC} to an small $\varepsilon > 0$). Note that these can be computed in practice efficiently — e.g., lb_{TEC} via an ILP of Benedikt et al. (2020) or by a branch and bound (Benedikt et al., 2026), whereas lb_{RCPSP} resembles a classical RCPSP problem that can be solved efficiently, e.g., by constraint programming (Heinz et al., 2025).

The objective function can therefore be written as:

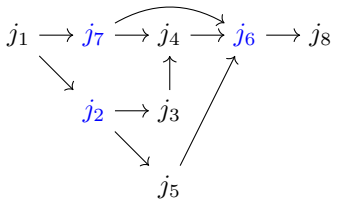
$$\min \left\{ \alpha \cdot \frac{1}{lb_{TEC}} \cdot \sum_{t \in \mathcal{I}} c_t P(\Omega_t) + (1 - \alpha) \cdot \frac{1}{lb_{RCPSP}} \cdot C_{\max} \right\} \tag{3.4}$$

One can observe that when $\alpha = 0$, the objective function reduces to calculating the makespan, whereas when $\alpha = 1$, it involves finding the best TEC within the given time horizon. Thanks to the normalization, when $\alpha = 0.5$, both objectives weigh the same in the optimal solution.

The problem can be modeled by the following ILP. Since this ILP models the entirety of our problem, we refer to it as the *monolithic* ILP. Let x_{jt} be a binary variable equal to 1 if the task $j \in \mathcal{T}$ starts at interval $t \in \mathcal{I}$, and z_{lm} be a binary variable indicating that the machine is performing an optimal transition between two processing states from time interval l to time interval m, except for intervals $l = 0$ and $m = h$ for which `off`state is considered instead of `proc`.

Decision Variables	
$x_{jt} = 1$	if the task $j \in \mathcal{T}$ starts at interval $t \in \mathcal{I}$, 0 otherwise
$z_{lm} = 1$	if the machine is performing an optimal SPACES transition between the intervals l and m (`proc` to `proc`), except for intervals $l = 0$ and $m = h$ for which `off` state is considered instead of `proc`, 0 otherwise

Table 4: Decision variables of the compact ILP.



(a) Task precedences (energy-intensive tasks in blue).

Params	j_1	j_2	j_3	j_4	j_5	j_6	j_7	j_8
p_j	2	1	2	1	3	2	2	2
r_0^j	0	1	0	0	0	1	1	0
r_1^j	5	0	2	3	2	0	0	3
r_2^j	2	0	1	1	2	0	0	2

(b) Task parameters.

Figure 3: Parameters for the example problem instance.

$$\min \frac{\alpha}{lb_{TEC}} \cdot \left(\sum_{j \in \mathcal{J}} \sum_{t=1}^{h} c_{jt}^{job} x_{jt} + \sum_{l=1}^{h} \sum_{m=1}^{h+1} c_{lm}^{*} z_{lm} \right) + \frac{(1-\alpha)}{lb_{RCPSP}} \cdot \left(\sum_{t=1}^{h} t x_{n+1,t} + 1 \right) \tag{3.5}$$

$$s.t. \sum_{t \in \mathcal{I}} x_{jt} = 1 \qquad \forall j \in \mathcal{T} \tag{3.6}$$

$$\sum_{j \in \mathcal{T}} \sum_{t'=\max\{1, t-p_j+1\}}^{t} r_k^j x_{jt'} \leq \rho_k \qquad \forall k \in \mathcal{R}, t \in \mathcal{I} \tag{3.7}$$

$$\sum_{t \in \mathcal{I}} t x_{j't} - \sum_{t \in \mathcal{I}} t x_{jt} \geq p_j \qquad \forall (j, j') \in A \tag{3.8}$$

$$\sum_{t \in \mathcal{I}} t x_{(n+1)t} - \sum_{t \in \mathcal{I}} t x_{jt} \geq p_j - 1 \qquad \forall j \in \mathcal{T} \tag{3.9}$$

$$\sum_{j \in \mathcal{J}} \sum_{t \in \text{StartTime} \cup \text{EndTime}_j} x_{jt} = 0 \tag{3.10}$$

$$\sum_{j \in \mathcal{J}} \sum_{t'=\max\{1, t-p_j+1\}}^{i} x_{jt'} + \sum_{l=1}^{t} \sum_{m=i+1}^{h+1} z_{lm} = 1 \qquad \forall t \in \mathcal{I} \tag{3.11}$$

where $c_{jt}^{job} = \sum_{k=t}^{t+p_j-1} c_k \cdot P(\texttt{proc}, \texttt{proc})$ is the cost of starting the job $j \in \mathcal{J}$ at interval $t \in \mathcal{I}$, and c_{lm}^{*} is the cost of the optimal transition from interval l to interval m (see Benedikt et al. (2020) for more details). As in the classical RCPSP, we introduce a dummy task $n+1$ as the last activity, representing the project's makespan.

Constraints (3.6) force all tasks to be performed once. Constraints (3.7) ensure that no resource capacity is exceeded. Constraints (3.8) enforce the precedence relations (3.1) between tasks, while constraints (3.9) calculate the makespan. Constraints (3.10) prevent energy-intensive tasks from being processed at an infeasible moment regarding machine transition. Finally, constraints (3.11) signify that at each time interval, the machine is either processing a task (that is in `proc`, indicating by the fact that the processing of the task started in the $\max\{1, i - p_j + 1\}$ previous intervals) or undergoing an optimal transition.

Example. Consider an instance with three resources $\{R_0, R_1, R_2\}$ and eight tasks $\mathcal{T} = \{j_1, \ldots, j_8\}$ with a subset $\mathcal{J} = \{j_2, j_6, j_7\}$ of energy-intensive tasks. The precedence constraints are shown in Figure 3a, the processing times and the resource capacities of each task are detailed in Figure 3b. We assume a transition diagram of the resource R_0 as the one shown in Figure 2. The optimal solution for the instance considered for $\alpha = 0.75$ is depicted in Figure 4.

Model motivation. The motivation for the scheduling model proposed in this work stems from manufacturing processes with energy-intensive production steps. For example, in glass production (Módos et al., 2016), the process is divided into two main phases: (i) preprocesing, and (ii) tempering. In the preprocessing steps, the glass panels are cut and drilled, while in the tempering stage, the preprocessed glass panels are heated in a furnace up to 620 °C and then cooled down (Módos et al., 2016). After that, other parts of the production process follow, such as polishing, quality control, and expedition. The furnace is modeled as a unit-capacity resource. An operation on this resource can represent a batch of multiple glass panels that are collected into a single operation via precedence constraints. The products in the furnace undergo a common thermal cycle — no products can enter or leave until the current batch is completed. The glass furnace is an expensive

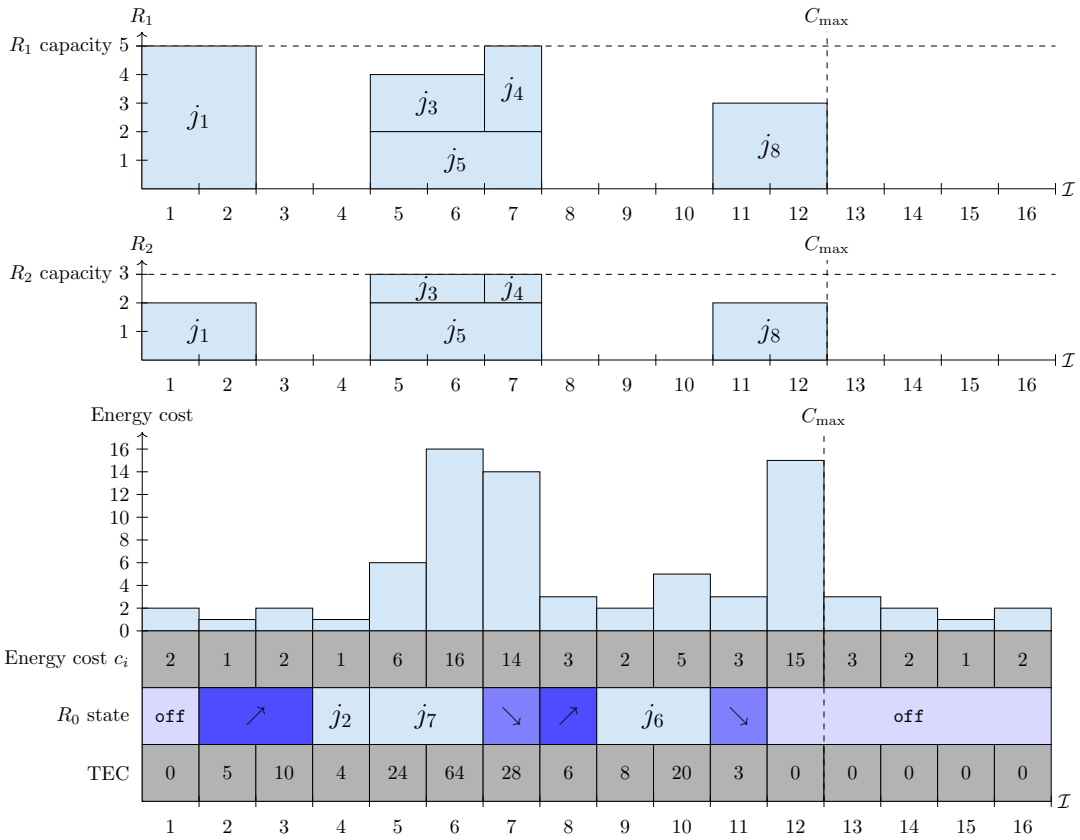


Figure 4: Example project schedule with the optimal transition for R_0. Parameter $\alpha = 0.75$ (more towards TEC): $C_{\max} = 12$, TEC $= 172$.

device; therefore, only a single one is available. Also, we assume that the considered problem represents an operational-level scheduling with much information already available, i.e., TOU prices are fully known for the upcoming near-term horizon (Bokde et al., 2021).

4. Constraint programming model

In this section, we propose a monolithic CP approach to solve the problem defined in the previous section. In particular, we will take advantage of the fact that in the CP, some variables may or may not be present in the final solution, depending on the value of other variables. These variables are called *"optional"* variables. Moreover, we remind the reader that in CP, an interval variable may have either a *fixed* length or be in a range of valid lengths. We represent the optimal transitions between the energy-intensive tasks with variables of a fixed length that will be allowed to move around the schedule as necessary. This trick allows us to reduce the number of variables needed to represent the optimal transitions, as we can precisely compute the maximum number of optimal transitions of each length that can be used in the schedule. In the remainder of the article, we will use the CP notations presented in Heinz et al. (2022).

Variables. We denote by ξ_j the interval variable of a length p_j associated with each task $j \in \mathcal{T}$. This variable corresponds to the processing of a task j, starting at time $\textsc{StartOf}(\xi_j)$ and ending at time $\textsc{EndOf}(\xi_j)$. By definition, each interval variable must be scheduled. We introduce optional interval variables ζ_{lk} to represent the optimal transition in the SPACES surrounding task processing on resource R_0. Let $l_{\max} = h - \sum_{j \in \mathcal{J}} p_j$ be the upper bound on the optimal transition length. The function $\mathrm{K} : l \mapsto \left\lfloor \frac{h - \sum_{j \in \mathcal{J}} p_j}{l} \right\rfloor$ then gives the upper bound on the number of optimal transitions of length l that can appear in the schedule. Therefore, the optional variables ζ_{lk} are created such that:

$$\textsc{LengthOf}(\zeta_{lk}) = l \quad \forall l \in \{1, \ldots, l_{\max}\}, k \in \{1, \ldots, \mathrm{K}(l)\} \tag{4.1}$$

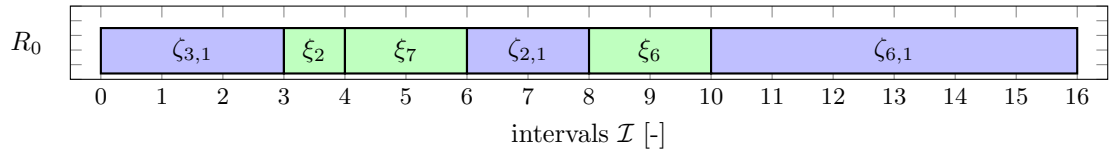


Figure 5: Example of interval variables present on resource R_0 in a valid schedule.

Figure 5 displays interval variables of the optimal solution of an instance comprising three energy-intensive tasks. Notice that the length of an absent interval always evaluates to 0.

Objective. We use a vector c_j^{job} to express the TEC for the processing of task j based on the interval i in which it starts. Additionally, since the variables representing the optimal switching are not fixed in their position, they are not directly associated with the total energy cost of the switching. Hence, we define the vector c_l^{space}, the ith element of the vector corresponds to the total energy cost of the optimal length switching l, if started in the interval i. The values of c_l^{space} are given as follows:

$$c_l^{space}[i] = c^*(i, i+l) \quad \forall i \in \{1, \ldots, h-l+1\}$$

The model minimizes the following objective:

$$\frac{\alpha}{lb_{TEC}} \cdot \left(\sum_{l=1}^{l_{\max}} \sum_{k=1}^{\mathrm{K}(l)} \textsc{Element}\left(c_l^{space}, \textsc{StartOf}(\zeta_{lk})\right) + \sum_{j \in \mathcal{J}} \textsc{Element}\left(c_j^{job}, \textsc{StartOf}(\xi_j)\right) \right) + \frac{1-\alpha}{lb_{RCPSP}} \cdot \max_{i \in \mathcal{T}} \{\textsc{EndOf}(\xi_i)\} \tag{4.2}$$

with the first part representing the TEC part of the objective and the second part representing the $C_{\max}$ part of the objective. Note that when computing the TEC, we consider only tasks requiring energy consumption in the set $\mathcal{J}$, while for the objective $C_{\max}$, we consider all the tasks.

Constraints. A solution is feasible if it respects the following constraints:

$$\textsc{StartOf}(\zeta_{lk}) \geq 0 \quad \forall l \in \{1, \ldots, l_{\max}\} \quad \forall k \in \{1, \ldots, \mathrm{K}(l)\} \tag{4.3}$$

$$\textsc{EndOf}(\zeta_{lk}) \leq h \quad \forall l \in \{1, \ldots, l_{\max}\} \quad \forall k \in \{1, \ldots, \mathrm{K}(l)\} \tag{4.4}$$

$$\textsc{PresenceOf}(\zeta_{lk}) \geq \textsc{PresenceOf}(\zeta_{lk+1}) \quad \forall l \in \{1, \ldots, l_{\max}\}, k \in \{1, \ldots, \mathrm{K}(l)-1\} \tag{4.5}$$

$$\textsc{EndBeforeStart}(\zeta_{lk}, \zeta_{lk+1}) \quad \forall l \in \{1, \ldots, l_{\max}\}, k \in \{1, \ldots, \mathrm{K}(l)-1\} \tag{4.6}$$

Constraints (4.3) and (4.4) ensure that all optimal transition variables, if present, are scheduled within the scheduling horizon. Constraints (4.5) and (4.6) give an order to the variables that represent the optimal transitions and ensure that the variables are used in an order given by their indexes for transitions of the same length. This removes redundant solutions that differ only in the order of variable use.

$$\textsc{StartOf}(\xi_j) \geq 1 + T(\texttt{off}, \texttt{proc}) \quad \forall j \in \mathcal{J} \tag{4.7}$$

$$\textsc{EndOf}(\xi_j) \leq h - T(\texttt{proc}, \texttt{off}) - 1 \quad \forall j \in \mathcal{J} \tag{4.8}$$

$$\textsc{EndOf}(\xi_j) \leq h \quad \forall j \in \mathcal{T} \tag{4.9}$$

$$\textsc{NoOverlap}\left(\{\xi_j : \forall j \in \mathcal{J}\} \cup \{\zeta_{lk} : \forall l \in \{1, \ldots, l_{\max}\}, k \in \{1, \ldots, \mathrm{K}(l)\}\}\right) \tag{4.10}$$

Constraints (4.7) and (4.8) ensure that all energy-intensive tasks are scheduled so that energy-intensive machines have time to warm up and shut down, respectively. Constraints (4.9) ensure that all tasks are scheduled within the scheduling horizon. Compared to the constraints (4.7) and (4.8), we consider all of the tasks in this case. NoOverlap constraints (4.10) guarantee that the resource R_0 never processes a task and

Algorithm 1 Logic-Based Benders Decomposition (LBBD)

while $Master$ is not optimal **do**
  $\hat{\boldsymbol{x}} \leftarrow Master()$ ▷ Get integer solution.
  $\hat{\boldsymbol{\zeta}} \leftarrow Subproblem(\hat{\boldsymbol{x}})$ ▷ Solve the subproblem.
  if $\hat{\boldsymbol{\zeta}}$ optimal or feasible **then**
    Add optimality cut (5.20) to $Master$ ▷ Give feedback to the $Master$
  else
    Compute $\mathcal{I}nf$ ▷ Retrieve tasks of $\mathcal{J}$ that lead to infeasibility.
    Add feasibility cut (5.17) to $Master$ ▷ Cut all solutions sharing same $\mathcal{I}nf$ at once.
  end if
end while

undergoes an optimal transition at the same time.

$$\sum_{l=1}^{l_{\max}} \sum_{k=1}^{\mathrm{K}(l)} \textsc{LengthOf}(\zeta_{lk}) = l_{\max} \tag{4.11}$$

$$\sum_{j \in \mathcal{T}} \textsc{Pulse}(\xi_j, r_\kappa^j) \leq \rho_\kappa \quad \forall \kappa \in \mathcal{R} \tag{4.12}$$

$$\textsc{EndBeforeStart}(\xi_i, \xi_j) \quad \forall (i,j) \in A \tag{4.13}$$

Constraint (4.11) ensures that the length of all optimal transitions used in the solution is exactly equal to the value of $l_{\max}$; optional variables that are not present in the solution do not count towards this sum. By the definition of $l_{\max}$, these constraints, together with constraints (4.10), also ensure that for the resource R_0, the whole scheduling horizon is filled by the processing of tasks or by optimal transitions in the SPACES around. Constraints (4.12) guarantee that resource capacities are not exceeded in any time interval. Finally, with A being the set of arcs of the precedence graph, Constraints (4.13) rule out any solutions in which the precedence relations are violated.

5. Logic-Based Benders Decomposition

This section introduces an LBBD approach for the problem considered, starting from its simplified version when $\alpha = 1$, to a general version for any $\alpha < 1$. The idea behind this approach is to separate the variables and constraints associated with the RCPSP from those related to the machine states problem. For that, first, we will not consider the makespan in the objective. Therefore, the RCPSP part can be seen as a feasibility problem inside the TEC with TOU tariffs and machine states on a single machine. We then divide the problem into two dependent problems. The master one will be the machine state problem, solved by the ILP-SPACES model, while the subproblem will be an RCPSP feasibility problem modeled by a CP formulation. Of course, this would only work if $\alpha = 1$. Therefore, we first present the special case $\alpha = 1$ in Section 5.1. Then, in Section 5.2, we will investigate how to generalize this approach to the case $\alpha < 1$. Note that the case $\alpha = 0$ is not interesting and therefore can be ignored. Indeed, as long as the constraints (3.10) are satisfied, the machine state problem will not generate further infeasibility, since the machine can remain in `proc` state for the duration of the schedule. That means that such a problem can be reduced to a classical RCPSP with a single time window for energy-intensive tasks whose size should be nearly equal to the time horizon. The layout of our LBBD approach is detailed in Algorithm 1.

5.1. RCPSP seen as a feasibility problem: Case $\alpha = 1$

In this section, we assume that $\alpha = 1$, i.e., that only the total energy part of the objective function (3.4) matters. Therefore, the RCPSP can be seen as a feasibility problem in which one must verify that all of the non-energy-intensive tasks can be scheduled in parallel to the energy-intensive ones, while not exceeding the time horizon.

5.1.1. Master problem

The master problem consists of optimizing the machine state problem alone, i.e., finding a scheduling of the energy-intensive tasks that minimizes the energy costs. Therefore, we keep in mind all the constraints of the ILP related to energy-intensive tasks. The decision variables presented in Table 4, are similar to those of the ILP, except that only the energy-intensive tasks are represented by the variables x_{jt}, i.e., $\mathcal{X} = \{x_{jt} \mid j \in \mathcal{J}, t \in \mathcal{I}\}$. All of the decision variables are binary. Hence, constraints (3.6), (3.8), (3.10), (3.11) and (3.7) (the latter limited to resource R_0) remain in the master. Constraints (3.9) are discarded as the makespan is no longer considered.

We denote the relation '$\prec$' for any tuple $u, v \in \mathcal{T}$ such that $u \prec v$ implies that u is a direct predecessor of v. We denote $Pred(j) \subset \mathcal{T}$ and $Succ(j) \subset \mathcal{T}$, the sets of predecessors and successors of a task $j \in \mathcal{T}$, respectively. The precedence constraints (3.8) cannot be used directly; indeed, since the binary variables x_{ji} represent only energy-intensive tasks, if we consider $u, v \in \mathcal{J}$ and $j \in \overline{\mathcal{J}}$, then precedence constraints such as $u \prec j \prec v$ would not be taken into account. Furthermore, we know that in this case $S_v \geq C_u + p_j$. Thus, for all $(u, v) \in \mathcal{J}^2$, if there exists a path $L_{uv} = \{j_1, \ldots, j_l\}$ such that $u \prec j_1 \prec \cdots \prec j_l \prec v$, then:

$$S_v \geq C_u + \sum_{j \in L_{uv}} p_j \tag{5.1}$$

Let $\mathcal{L}_{uv}$ denote the set of all possible precedence paths from task u to task v, i.e., if there exist L possible precedence paths between u and v, then $\mathcal{L}_{uv} = \{L^l_{uv}\}_{l \in \{1,\ldots,\lambda\}}$. This set is finite because there is a finite number of tasks. We say that if $\mathcal{L}_{uv} \neq \emptyset$, then u is an ancestor of v, and v is a descendant of u. We name L^*_{uv} the longest path in $\mathcal{L}_{uv}$. Then, we can state that:

Proposition 1. *For any $(u, v) \in \mathcal{J}^2$, if $\mathcal{L}_{uv} \neq \emptyset$, then:*

$$S_v \geq C_u + \sum_{j \in L^*_{uv}} p_j \tag{5.2}$$

Proof. Consider a pair $(u, v) \in \mathcal{J}^2$ such that $\mathcal{L}_{uv} \neq \emptyset$. Then, there exist paths $L^1_{uv}, \ldots, L^\lambda_{uv}$ in $\mathcal{L}_{uv}$. Without loss of generality, we can reorder these paths so that:

$$\sum_{j \in L^1_{uv}} p_j \leq \sum_{j \in L^2_{uv}} p_j \leq \cdots \leq \sum_{j \in L^\lambda_{uv}} p_j$$

Then, according to (5.1), we have:

$$C_u + \sum_{j \in L^1_{uv}} p_j \leq \cdots \leq C_u + \sum_{j \in L^\lambda_{uv}} p_j \leq S_v$$

As $L^*_{uv} = L^\lambda_{uv}$ holds by construction, the statement follows. □

We introduce the function MD : $u \times v \in \mathcal{J}^2 \to \mathbb{Z}^+ \cup \{\infty\}$ such that MD(u, v) is the minimal processing time between tasks u and v if v is a descendant of u, otherwise ∞, i.e.,

$$\forall u, v \in \mathcal{J},\ \text{MD}(u, v) = \begin{cases} p_u + \sum_{j \in L^*_{uv}} p_j & \text{if } \mathcal{L}_{uv} \neq \emptyset \\ \infty & \text{otherwise} \end{cases} \tag{5.3}$$

The domain of MD can be extended to non-energy-intensive tasks $\overline{\mathcal{J}}$, which is useful for handling cases where all predecessors — or all successors — of an energy-intensive task j are themselves non-energy-intensive. Computing this function is equivalent to calculating the longest paths between all nodes of the precedence graph associated with the RCPSP. Since there is no cyclic relation (i.e., $\nexists j \in \mathcal{T},\ j \prec \cdots \prec j$) in the RCPSP, the precedence graph is a directed acyclic graph. Hence, the longest paths can be found by computing the shortest paths in the precedence graph in which each arc leaving a node j has a weight equal to $-p_j$. Furthermore, since the precedence graph is usually sparse, we use Johnson's all-pairs shortest path algorithm (Johnson, 1977), which has a complexity of $O(|V||A| \log |V|)$ in a sparse graph $G(V, A)$. We say that a task j is the *predecessor* (resp. *successor*) of a task i if MD$(i, j) < \infty$ (resp. MD$(j, i) < \infty$). In light of the above, the master ILP can

be written as:

$$(Master): \quad \min \sum_{j\in\mathcal{J}}\sum_{t=1}^{h} c_{jt}^{job} x_{jt} + \sum_{l=1}^{h}\sum_{m=1}^{h+1} c_{lm}^{*} z_{lm} \tag{5.4}$$

subject to

$$\sum_{t\in\mathcal{I}} x_{jt} = 1 \quad \forall j \in \mathcal{J} \tag{5.5}$$

$$\sum_{t\in\mathcal{I}} t x_{vt} - \sum_{t\in\mathcal{I}} t x_{ut} \geq \mathrm{MD}(u,v) \quad \forall (u,v) \in \mathcal{J}^2, \mathrm{MD}(u,v) < \infty \tag{5.6}$$

$$\sum_{j\in\mathcal{J}} \sum_{t'=\max\{1,t-p_j+1\}}^{t} x_{jt'} \leq 1 \quad \forall t \in \mathcal{I} \tag{5.7}$$

$$\sum_{j\in\mathcal{J}} \sum_{t\in \mathrm{StartTime}\cup\mathrm{EndTime}_j} x_{jt} = 0 \tag{5.8}$$

$$\sum_{j'\in Pred(j)\cup\overline{\mathcal{J}}} \sum_{t=0}^{\mathrm{MD}(j',j)} x_{jt} + \sum_{j'\in Succ(j)\cup\overline{\mathcal{J}}} \sum_{t=h-\mathrm{MD}(j,j')+1}^{h} x_{jt} = 0 \quad \forall j \in \mathcal{J} \tag{5.9}$$

$$\sum_{j\in\mathcal{J}} \sum_{t'=\max\{1,t-p_j+1\}}^{t} x_{jt'} + \sum_{l=1}^{t}\sum_{m=t+1}^{h+1} z_{lm} = 1 \quad \forall t \in \mathcal{I} \tag{5.10}$$

Constraints (5.6) are a modified version of constraints (3.8) guarantee that each couple $(u,v) \in \mathcal{J}^2$ satisfies Proposition 1. Constraints (5.9) address the scenarios where an energy-intensive task is preceded (resp. succeeded) by a sequence of non-energy-intensive tasks only. In this case, these constraints compel the energy-intensive task to allocate sufficient time before (resp. after) to allow this sequence to be scheduled. These two sets of constraints prune solutions that would have an associated subproblem that is trivially infeasible, without having the burden of verifying them. The remaining constraints are similar to the ILP constraints (although restricted to the set $\mathcal{J}$ instead of $\mathcal{T}$).

It is worth noting that an integer solution found by the master problem may not be a valid solution to the RCPSP, as the resource constraints of non-energy-intensive tasks may not necessarily be enforced. Hence, each time a feasible solution of the master problem is found, we must verify the feasibility for the RCPSP by means of an oracle that consists of solving the subproblem; if the oracle finds the master solution infeasible, new constraints are injected into the master to cut off this solution.

5.1.2. Subproblem

The subproblem consists of finding a feasible solution to the RCPSP problem associated with the solution provided by the master problem. Since $\mathcal{X}$ are fixed integers in the subproblem, they impose the starting time of the energy-intensive tasks in the RCPSP. They shall be considered as parameters in the subproblem, denoted as $\hat{\mathcal{X}} = \{\hat{x}_{jt}\}_{j\in\mathcal{J},t\in\mathcal{I}}$. The subproblem also needs to schedule all tasks before the time horizon. Given the findings of Heinz et al. (2025), which show that the constraint programming paradigm is highly effective in solving RCPSPs under a constraint time horizon, we adopt this methodology to address the subproblem. The proposed CP model is the following:

$$(Subproblem) \quad C_{\max}^{sub}: \min \max_{j\in\mathcal{T}} \textsc{EndOf}(\xi_j) \tag{5.11}$$

$$\sum_{j\in\mathcal{J}} \textsc{Pulse}(\xi_j, r_k^j) \leq \rho_k \qquad \forall k \in \mathcal{R} \tag{5.12}$$

$$\textsc{EndBeforeStart}(\xi_i, \xi_j) \qquad \forall (i,j) \in A \tag{5.13}$$

$$\textsc{EndOf}(\xi_j) \leq h \qquad \forall j \in \overline{\mathcal{J}} \tag{5.14}$$

$$\textsc{StartOf}(\xi_j) = t \qquad \forall j \in \mathcal{J}, t \in \mathcal{I}, \hat{x}_{jt} = 1 \tag{5.15}$$

$$\textsc{Interval}\ \xi_j,\ \textsc{LengthOf}(\xi_j) = p_j \qquad \forall j \in \mathcal{T} \tag{5.16}$$

As in the monolithic CP described in Section 4, constraints (5.12) ensure that the capacities of each resource are not exceeded. Constraints (5.13) model the precedence constraints. Constraints (5.14) prevent tasks from being scheduled after the end of the schedule horizon, while constraints (5.15) impose that energy-intensive tasks start at the same time as in the master solution. The Objective (5.11) minimizes the makespan. In the particular case $\alpha = 1$, this objective function (5.11) does not need to be added to the CP, since our subproblem is therefore considered a feasibility problem. However, at the end of the search, after an optimal solution has been found, it can be relevant to recompute its associated subproblem in view of the objective function, such that we could retrieve the optimal makespan of the master optimal solution (this is only for a practical perspective; it does not change anything on the optimal cost of energy).

Let us denote $\mathcal{X}^\dagger = \{x_{jt} \mid x_{jt} \in \mathcal{X}, \hat{x}_{jt} = 1\}$. When the subproblems cannot find a valid solution for a given energy-intensive task schedule given by the master, we compute the minimal infeasibility set $\mathcal{I}nf \subseteq \mathcal{X}^\dagger$ over the constraints (5.15). This set contains the variables appearing in the minimal conflict that causes the infeasibility. Since the conflict is minimal, the removal of any one of these constraints will remove that particular cause of infeasibility. Note that there may be other conflicts in the solution. This set can be automatically computed by the CP solver (Laborie et al., 2018) if it proves the infeasibility of the subproblem. If the solver cannot compute this subset in a reasonable amount of time, then, since we know the solution is infeasible, we set $\mathcal{I}nf = \mathcal{X}^\dagger$. Once this set is determined, the following cuts are added to the master.

Feasibility cuts. Given a minimal infeasibility set $\mathcal{I}nf$, we add in *Master* the following feasibility cut:

$$\sum_{x_{jt} \in \mathcal{I}nf} x_{jt} \leq |\mathcal{I}nf| - 1 \tag{5.17}$$

This cut enforces that all, but one, of the energy-intensive tasks involved in the infeasibility set can keep their current starting time in a feasible solution. This rules out the current infeasible solution while also pruning all solutions that share the same infeasible subset. If $\mathcal{I}nf = \mathcal{X}^\dagger$, only the solution defined by $\mathcal{X}^\dagger$ is cut off.

5.1.3. Warmstarts

An increase in instance size may undermine the efficacy of ILP approaches, thereby hindering their ability to find feasible solutions. In contrast, CP approaches are highly efficient in identifying feasible solutions quickly. However, these approaches often struggle to narrow the optimality gap due to their limited ability to generate strong lower bounds. Therefore, we propose two ways of generating a warmstart solution, both based on CP. The first uses the classical CP model (Artigues et al., 2013) with additional constraints (4.7) and (4.4). However, we do not provide any objective function to the CP solver; the objective is to find a valid solution. We call *fsws* this warmstart approach. The second solves the problem through the CP model delineated in Section 4, within a limited time frame of approximately one minute. We name *cpws* this warmstart approach. Subsequently, the optimal solution identified through these processes is utilized as a starting point for our ILP approaches, encompassing both ILP and LBBD.

5.2. RCPSP weighting on the master objective: Case $\alpha < 1$

Now, let us focus on the case $\alpha < 1$. In this case, the $C_{\max}$ objective function of the RCPSP also contributes to the objective function. Therefore, we need to introduce a way to represent the makespan within the master problem. The issue is that, since most of the tasks of the RCPSP are not present in the master, we cannot induce the makespan directly from it.

Therefore, we introduce a new decision variable $q \in \mathbb{Z}_{\geq 0}$ to represent the value of makespan. We rewrite the objective function as:

$$\min \ \frac{\alpha}{lb_{TEC}} \cdot \left(\sum_{j \in \mathcal{J}} \sum_{t=1}^{h} c_{jt}^{job} x_{jt} + \sum_{l=1}^{h} \sum_{m=1}^{h+1} c_{lm}^{*} z_{lm} \right) + \frac{1-\alpha}{lb_{RCPSP}} \cdot q \tag{5.18}$$

As described above, we require new constraints to ensure that the variable q accurately models the makespan, in the case where the subproblem is feasible. Indeed, if the subproblem is infeasible, feasibility cuts (5.17) are

added. Since the makespan is only known after computing the subproblem, we need Benders optimality cuts to inject the value of the optimal subproblem makespan $C^{sub}_{\max}$ into the master.

Optimality cut. Artigues et al. (2003) showed that a solution of an RCPSP can be modeled as a resource-flow graph. The critical path of this graph contains all the tasks that contribute directly to the makespan, i.e., any change in the starting time of one of these tasks would modify the makespan. The general idea of the proposed optimality cut is that, for a given solution of a problem, only a subset of the energy-intensive tasks are on the resource-flow critical path, so that only the tasks in this subset have to be in the optimality cut. In addition, by exploiting the precedence structure along this resource-flow critical path, we can estimate a lower bound on the makespan deviation. Let $\Pi^{\vartheta} = \{j_1, j_2, \ldots, j_{|\Pi^{\vartheta}|}\}$ be the critical energy-intensive tasks of an optimal subproblem solution ϑ, indexed in increasing order of start time, and let $\hat{t}^{\vartheta}_{j_k}$ denote the start time of j_k in ϑ. For each j_k we define:

$$\varphi(j_k, \hat{t}^{\vartheta}_{j_k}) \;=\; \max\Big\{\hat{t}^{\vartheta}_{j_l} + p_{j_l} \;\Big|\; l < k,\; \hat{t}^{\vartheta}_{j_l} + p_{j_l} \le \hat{t}^{\vartheta}_{j_k}\Big\}, \tag{5.19}$$

with the convention $\varphi(j_1, \hat{t}^{\vartheta}_{j_1}) = 0$. The quantity $\varphi(j_k, \hat{t}^{\vartheta}_{j_k})$ is the longest completion time, no later than $\hat{t}^{\vartheta}_{j_k}$, of a critical energy-intensive task of ϑ scheduled before j_k; it acts as the closest energy-intensive "anchor" on the chain leading to j_k, even when the immediate predecessor on the critical resource-flow path is itself non energy-intensive. Note that, by construction, $\varphi(j_k, \hat{t}^{\vartheta}_{j_k}) \le \hat{t}^{\vartheta}_{j_k} \le C^{sub}_{\max}$. The intuition for the cut is that, if the master deviates from the current critical assignment at $(j_k, \hat{t}^{\vartheta}_{j_k})$, the energy-intensive task of realizing $\varphi(j_k, \hat{t}^{\vartheta}_{j_k})$ is preserved in the subproblem and must still complete by that value, so the makespan cannot fall below it. We can therefore deduce the following Benders optimality cut:

$$C^{sub}_{\max} \;-\; \sum_{j_k \in \Pi^{\vartheta}} \big(C^{sub}_{\max} - \varphi(j_k, \hat{t}^{\vartheta}_{j_k})\big)\,(1 - x_{j_k, \hat{t}^{\vartheta}_{j_k}}) \;\le\; q. \tag{5.20}$$

When the set Π^{ϑ} is empty (no critical energy-intensive task could be identified on the resource-flow graph), the sum is taken over the entire set of energy-intensive tasks scheduled in ϑ, which conservatively recovers a valid cut.

Proposition 2. *Inequality* (5.20) *is a valid Benders optimality cut.*

Proof. To be a valid Benders optimality cut, the inequality must (i) cut off the current suboptimal master solution and (ii) preserve any other feasible integer solution. Let ϑ be the current suboptimal solution and denote

$$Q = \Big\{x_{j_k, \hat{t}^{\vartheta}_{j_k}} \;\Big|\; j_k \in \Pi^{\vartheta}\Big\}$$

the set of master assignments fixed by the critical energy-intensive tasks of ϑ. For any feasible master solution ϱ, let

$$R = \Big\{x_{j_k, \hat{t}^{\varrho}_{j_k}} \;\Big|\; j_k \in \Pi^{\vartheta}\Big\}$$

denote the placements taken by those same tasks in ϱ, and define

$$S^{\varrho} \;=\; \big\{\, j_k \in \Pi^{\vartheta} \;\big|\; \hat{t}^{\varrho}_{j_k} \neq \hat{t}^{\vartheta}_{j_k} \,\big\}$$

so that $S^{\varrho} = \emptyset$ if and only f $Q \cap R = Q$. We distinguish two cases.

Case $Q \cap R = Q$ (i.e., $S^{\varrho} = \emptyset$): every factor $(1 - x_{j, \hat{t}^{\vartheta}_j})$ vanishes and (5.20) reduces to $C^{sub}_{\max} \le q$, which is correctly enforced since reproducing the critical chain of ϑ forces $C^{\varrho}_{\max} \ge C^{\vartheta}_{\max} = C^{sub}_{\max}$.

Case $Q \cap R \neq Q$ (i.e., $S^{\varrho} \neq \emptyset$): let $j_{k^\star}$ be the element of S^{ϱ} of smallest index, that is, the earliest critical energy-intensive task of ϑ whose start time is changed in ϱ. By minimality of $k^\star$, every $j_l \in \Pi^{\vartheta}$ with $l < k^\star$ satisfies $\hat{t}^{\varrho}_{j_l} = \hat{t}^{\vartheta}_{j_l}$. In particular, the energy-intensive task achieving the maximum in the definition of $\varphi(j_{k^\star}, \hat{t}^{\vartheta}_{j_{k^\star}})$ is placed identically in ϱ, hence still completes at time $\varphi(j_{k^\star}, \hat{t}^{\vartheta}_{j_{k^\star}})$ and is preserved by the subproblem. Therefore,

$$C^{\varrho}_{\max} \;\ge\; \varphi(j_{k^\star}, \hat{t}^{\vartheta}_{j_{k^\star}}). \tag{5.21}$$

Evaluating the left-hand side (LHS) of (5.20) at ϱ and using that each coefficient $(C^{\vartheta}_{\max} - \varphi(j_l, \hat{t}^{\vartheta}_{j_l}))$ is non-negative,

$$C^{\vartheta}_{\max} \; - \sum_{j_l \in S^{\varrho}} \left(C^{\vartheta}_{\max} - \varphi(j_l, \hat{t}^{\vartheta}_{j_l})\right) \; \leq \; C^{\vartheta}_{\max} - \left(C^{\vartheta}_{\max} - \varphi(j_{k^\star}, \hat{t}^{\vartheta}_{j_{k^\star}})\right) \; = \; \varphi(j_{k^\star}, \hat{t}^{\vartheta}_{j_{k^\star}}),$$

so combining with (5.21) we obtain $q = C^{\varrho}_{\max} \geq$ LHS of (5.20), as required. Hence (5.20) cuts off only suboptimal master solutions and preserves all feasible ones. □

Valid inequalities. The optimality constraints update the value of the objective of the master problem. However, they are only evaluated for integer solutions. To reduce the master search tree, it is worth noting that it is possible to estimate a lower bound on the makespan based on the release date of a task.

Proposition 3. *For every energy-intensive task $j \in \mathcal{J}$, the makespan q must satisfy the following constraints:*

$$\sum_{t \in \mathcal{I}} t x_{jt} + \mathrm{MD}(j, s) + p_s \leq q \quad \forall s \in Succ(j) \tag{5.22}$$

Proof. The makespan of a solution of our problem cannot be less than the minimum amount of time required to process the consecutive tasks, i.e., as if the resource constraints of the RCPSP were ignored and only precedence constraints were present. Thus, we can deduce that, for each energy-intensive task j, a valid makespan must be at least equal to the largest minimum distances between j and its successors, i.e.:

$$\forall j \in \mathcal{J}, \forall s \in Succ(j), \; C_{\max} \geq S_j + \mathrm{MD}(j, s) + p_s$$

Hence, we can infer the constraints (5.22) from them. □

Let us define $\mathrm{MD}^*(j) = \max_{s \in Succ(j)}\{\mathrm{MD}(j, s)\}$ the *maximal minimal distance* over the successors of a task $j \in \mathcal{T}$.

Corollary 1. *For every energy-intensive task $j \in \mathcal{J}$, the makespan q must satisfy the following constraint:*

$$q \geq \sum_{t \in \mathcal{I}} t x_{jt} + \mathrm{MD}^*(j) \tag{5.23}$$

Constraints (5.23) allow us to compute a lower bound for the makespan in the master problem, considering the starting times of the energy-intensive tasks. They greatly help the convergence of our LBBD. Indeed, Constraints (5.20) ultimately only come into play to correct an inaccurate estimate of the makespan, in cases where constraints in resource capacity prevent tasks from being completed as quickly as they could.

Also, we know that the makespan of an optimal solution of our problem cannot exceed by construction the lb_{RCPSP} makespan. Hence, we can add the following constraint:

$$q \geq lb_{RCPSP} \tag{5.24}$$

This constraint (5.24) contributes significantly to the convergence of our solution (see Figure 6); indeed, it provides a strong assumption on the value of q, which helps to shrink the search space. In fairness, we also inject this constraint into the monolithic formulations.

Additional remarks. To conclude this section, we discuss an important remark. The proposed LBBD guarantees the optimality of the final solution as long as all the subproblems have been either solved to optimality or proved infeasible. However, depending on the size or the complexity of the subproblem, it may end up being hard to prove either the feasibility or the optimality of *Subproblem* in a reasonable time. In these cases, we proceed as follows: if *Subproblem* finds a feasible solution but cannot close the optimality gap, we insert in *Master* an optimality constraint (5.20) where $C^{sub}_{\max}$ is the best makespan found so far. On the contrary, when *Subproblem* can neither identify a feasible solution nor assert that no solution exists, we consider that the problem is infeasible, and we rule out the solution with a feasibility cut (5.17) with $\mathcal{I}nf = \mathcal{X}^{\dagger}$. Alas, both situations may lead to optimal solutions being cut off in the master. Although CP approaches are known to be efficient in

deciding feasibility, they may struggle to close the gap between the lower and upper bounds. To tackle this issue that could impede the validity of our approach, we divide the experimental sections into two sections. In the first one, we will only consider instances in which the LBBD can certify the optimality of the provided solutions, i.e., all subproblems have either been solved to optimality or declared infeasible. In this section, the LBBD can thus be treated as an optimal method. We denote it `LBBD-opt`. In the next section, we will impose a time limit on the computation of the subproblem. By doing so, we may end up generating suboptimal cuts. We denote it `LBBD-heur`.

6. Computational experiments

In this section, we assess to what extent the proposed LBBD is a practical alternative to the two monolithic baselines, the compact ILP of Section 3 and the CP model of Section 4, and we identify the regimes, in terms of instance density and weighting parameter α, in which one approach is preferable to another. Throughout the section, we compare four methods. The monolithic ILP warm-started from a feasible solution of the underlying RCPSP (found via a classical CP formulation of RCPSP), denoted `ILP-fsws`, serves as the reference, and the per-instance gain δ_μ for any other method μ is calculated as a percentage according to Equation (E.1). The three remaining methods are: the same monolithic ILP warm-started from a 60 seconds run of `freecp` solution (`ILP-cpws`), the monolithic constraint-programming model (`freecp`), and the proposed LBBD detailed in Section 5, denoted `LBBD`.

The experiments are divided into three sections. After detailing the dataset, we investigate the impact of each `LBBD`'s components on the solution quality. Then, we compare our novel approach to the others over instances in which `LBBD` guarantees the optimality of its optimal solution, whereas in the last section, we observe the `LBBD` over all instances. Finally, we synthesize the results and critically evaluate the relevance and performance of each method with respect to the specific instance configurations.

All experiments were run on 2 x AMD™ EPYC™ 9124 CPUs 3.5 GHz equipped with 64 threads in total and 384 GB of RAM, running NixOS 25.05. Gurobi 13.0.1 and IBM CP Optimizer 22.1.1.0 are used to solve the ILP and CP models, respectively. All the code has been written in `C++23`, compiled with Clang 19.1.7 and the `O2` optimization flag, using the Boost Graph Library (Siek et al., 2001) to implement graph components. All experiments are run on a single thread, and the solver's default pre-solve strategies are enabled. A time limit of 1800 seconds is imposed for all experiments. We impose a soft limit on the memory usage of the solvers of 50 GB. The other solver parameters are left at their default values, except for Gurobi's numeric focus set to 2. In addition, the precomputation time of the paths SPACES graph and of the MD function (5.3) are not reported in our experiments, because they never exceed one second in every instance of our datasets, which is negligible.

6.1. Datasets

We generate our instances based on the RCPSP instances of the standard PSPLIB dataset (Sprecher and Kolisch, 1996) that contain sets of instances with $\{30, 60, 90, 120\}$ activities. To increase the number of tasks in our instances, we defined an approach to merge multiple small RCPSP instances from PSPLIB until we obtained an instance of the desired size[1]. When we merge two smaller instances, the resources' capacities are set to the maximum of the two. Then, these two merged projects are connected to the dummy start and end tasks, creating a parallel network of the two projects. The process continues until the requested instance size is met (or exceeded). In this way, the complexity of the original RCPSP instances is preserved. See an example of the precedence structure in the instances generated in the supplementary materials. We consider for each instance five resources $(R_0, \ldots, R_4)$. Then, we selected a given percentage of tasks in the newly created instance that we marked as "energy-intensive" (i.e., we set all of its other resource requirements to 0, then we added a requirement of 1 for R_0). These tasks are selected uniformly throughout the project. Indeed, a more in-depth analysis revealed no significant effect on the allocation or positioning of energy-intensive tasks within the

[1]The generator and the instances are available in github.com/corentinjuvigny/rcpsp_ms_tou_generator

project workflow. The energy-intensive resource R_0 uses a transition diagram from Figure 2, which is based on a diagram proposed by Shrouf et al. (2014). Finally, we generate a cost vector c of energy prices based on real TOU data collected from the Czech OTE[2]. We partition our test instances according to their ratio $\rho := |\mathcal{J}|/|\mathcal{T}|$ into three categories:

- Sparse: $\rho \approx 3$ - 5%
- Standard: $\rho \approx 15$ - 20%
- Dense: $\rho \approx 50\%$

For each of those categories, we generated instance sets of increasing sizes. Each set contains 20 instances. Sparse instances have up to 640 tasks, standard instances up to 480 tasks, and dense instances up to 224 tasks. Each instance has its own distinct cost vector. Tables C.1 and C.2 of the supplementary materials provide additional details on the number of tasks and the time horizon for each set. The datasets containing the sparse, standard, and dense instance sets are available at [dataset] Juvigny (2025e), [dataset] Juvigny (2025d), and [dataset] Juvigny (2025a), respectively.

6.2. Analysis of `LBBD` components

First, we analyze the components of the proposed `LBBD` approach to assess their impact on the overall quality of the method. We do all the experiments on the six smallest standard sets. Only instances for which the `LBBD-heur` guarantees optimality of its solutions are considered in this paragraph.

Impact of global lower bound on makespan (constraint (5.24)*).* First, let us analyze the impact of introducing a global lower bound on the makespan. Figure 6 compares the computation time between `LBBD-opt` with (*`LBBD-lbq`*) and without (*`LBBD-nolbq`*) the lower bound on the makespan. We see that the closer α is to 0, the more impactful the lower bound is on the convergence speed of our approach, which is negligible when $\alpha = 1$. The behavior is reasonable because the smaller α, the greater the contribution of the makespan to the objective value. Table 5 corroborates the analysis by showing that the approach without constraint 5.24 requires many more Benders cuts than the approaches with it, especially when α is close to 0. In fairness, this global lower bound is also added in ILP formulations.

α	method	Avg optimality cuts	Avg feasibility cuts
0.25	`LBBD-nolbq`	1306.97	26.30
	`LBBD-lbq`	405.12	15.50
	`LBBD-lbq-cpws`	106.17	10.07
0.50	`LBBD-nolbq`	192.65	13.03
	`LBBD-lbq`	56.13	4.88
	`LBBD-lbq-cpws`	16.98	3.20
0.75	`LBBD-nolbq`	24.08	1.27
	`LBBD-lbq`	21.72	1.35
	`LBBD-lbq-cpws`	10.98	0.67
1.00	`LBBD-nolbq`	8.98	1.62
	`LBBD-lbq`	8.98	1.62
	`LBBD-lbq-cpws`	5.42	1.23

Table 5: Average number of optimality and feasibility cuts per `LBBD-opt` variant on the reduced sets of standard instances.

Impact of the `freecp` warmstart procedure. Figure 6 also provides the computation time performance of the `LBBD-opt` warmstarted by a solution found by a 60s run of `freecp` (*`LBBD-lbq-cpws`*). Clearly, we see that during the first 60 seconds, the latter proves the optimality of only a few instances. Indeed, this time interval corresponds to the computation of the warmstart solution by `freecp`. But once this warmstart solution yielded, the percentage of solutions proven to be optimal skyrockets, surpassing the percentage of the non-CP-warmstarted

[2]https://www.ote-cr.cz/en

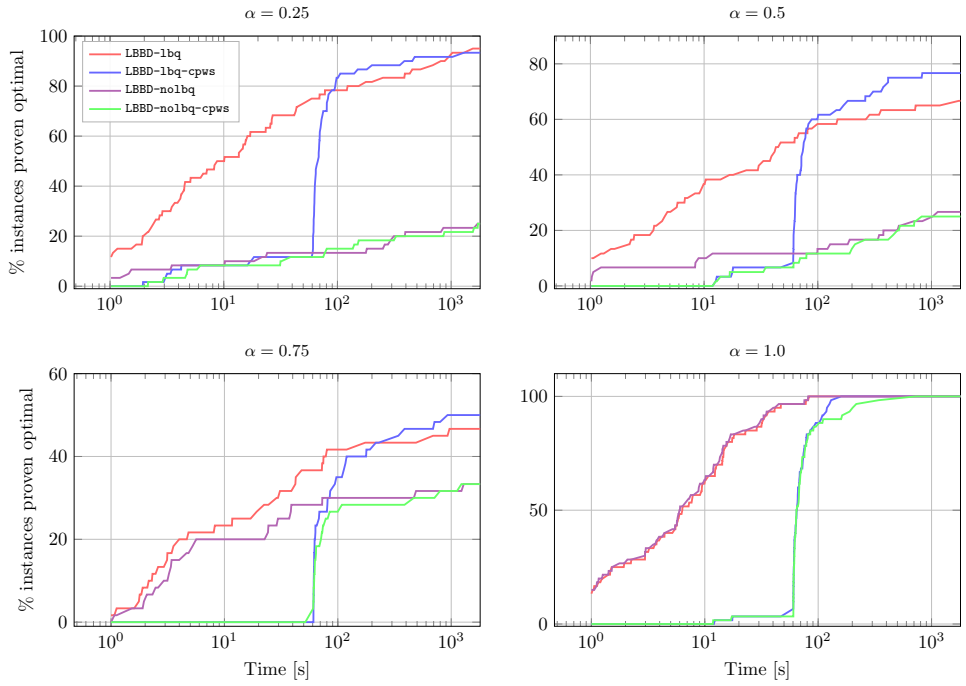


Figure 6: Comparison of the percentage of solutions solved to optimality in less than t seconds between the `LBBD-opt` with (*`LBBD-lbq`*) / without (*`LBBD-nolbq`*) constraint (5.24) and the warmstarted with a 60s-timelimit solution of `freecp` (*`LBBD-lbq-cpws`* / *`LBBD-nolbq-cpws`*).

Instance	sparse_5_5			sparse_10_3		
Sub-TL	obj	Time (s)	#Non-opt	obj	Time (s)	#Non-opt
1	1.12471	155	93	1.07633	1739.22	1744
5	1.12077	400	67	1.07804	1843.72	368
10	1.11915	731	66	1.07368	1858.45	185
15	1.11915	1065	66	1.07938	1847.40	122
20	1.11915	1421	67	1.07938	1915.40	95
30	1.11915	2100	67	1.08874	1944.12	63

Table 6: Evolution of the objective and computation time of two sparse instances when time limit of the subproblem changes ($\alpha = 0.5$).

version, showing that this warmstart solution is a good starting point. In addition, Table 5 indicates that the CP-warmstarted version requires fewer Benders cuts, which means that it explores fewer integer solutions. Moreover, Figure 6 shows that the impact of the constraint (5.24) on the convergence time is greater than the warm starting, since *`LBBD-nolbq-cpws`* does not noticeably provide better results than *`LBBD-nolbq`*.

Time limits tuning. The time limit of the subproblem influences the performance of the `LBBD`. If set too tight, we may rule out an optimal solution, but set too long, it may impede the overall search by allotting too much time in closing the gap of one subproblem instead of exploring other ones. Table 6 shows, for two sparse instances (in which the time limit of the subproblem is critical), that by increasing the time limit the objective value decreases until it reaches a plateau from which it goes back up. We employ iRace (López-Ibáñez et al., 2016) to find adequate values of the time limits of *Subproblem* and `freecp` warmstart that maximize the ratio of the quality of the solution found over time consumption. The results gave a warmstart computation time limit of 60s and a subproblem computation time of 20s.

In summary, the additional procedures and constraints proposed significantly improve our LBBD method. Therefore, we will henceforth consider that they are included in our LBBD method.

6.3. Comparison `LBBD-opt` — `ILP` — `freecp`

In this subsection, we will only consider instances for which the LBBD approach guarantees the optimality of its optimal solution, i.e., all of its subproblems have been either solved to optimality or proved infeasible.

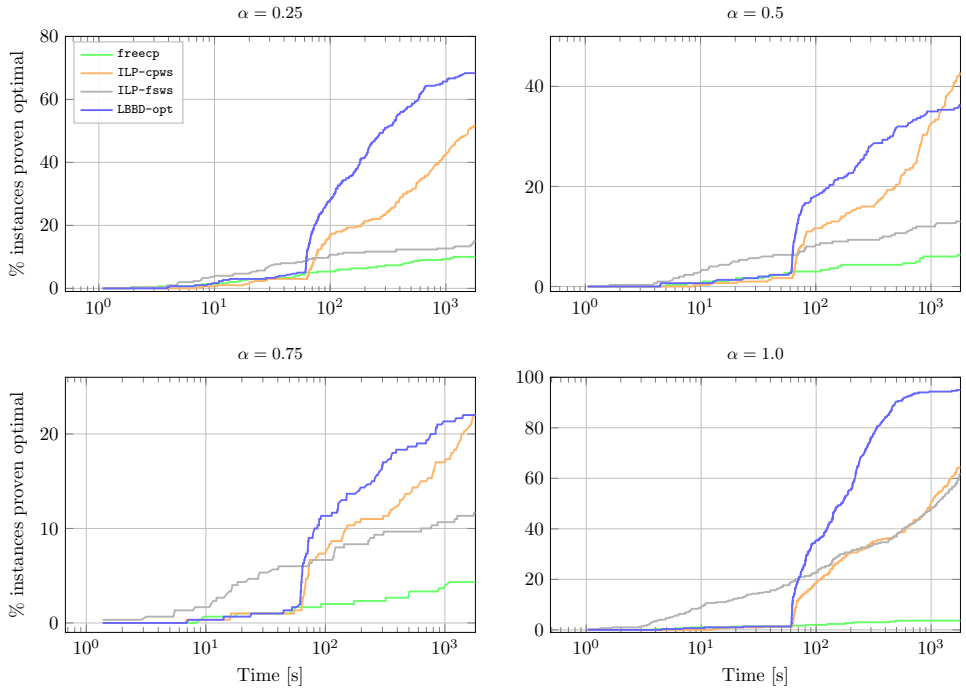


Figure 7: Comparison of the percentage of solutions solved to optimality over time for each method in standard sets.

We propose to compare the following approaches: the monolithic ILP formulation, warm-started from a feasible solution (*ILP-fsws*), the monolithic ILP warm-started from a 60s-`freecp` solution (*ILP-cpws*), the monolithic CP approach (*freecp*), and the proposed Benders decomposition approach (*LBBD-opt*). The `ILP-fsws` is our reference method, i.e., for any other method μ, we compute the delta of this reference method as:

$$\delta_\mu = \frac{obj_{\texttt{ILP-fsws}} - obj_\mu}{|obj_{\texttt{ILP-fsws}}|} \cdot 100 \tag{6.1}$$

Additional graphics, including box plots and more detailed tables, are available in the supplementary materials.

Standard instances. Figure 7 shows the percentage of instances of standard density over time for each method. It reveals that the `LBBD-opt` approach outperforms the others by proving the optimality of many more instances, and the advantage is most striking at $\alpha = 1$, where it proves close to 95% of the instances optimal against roughly 65% for `ILP-cpws` and below 5% for `freecp`. As is typical for a pure CP model, `freecp` proves the fewest instances optimal of all four methods. When we observe the quality of the solutions that each method produces (Table 7, measured by the gain (E.1) over `ILP-fsws`), we see that `LBBD-opt` achieves better gains than the other methods when $\alpha \in \{0.25, 1.0\}$ and is slightly better than `freecp` in the biggest set of instances when $\alpha \in \{0.5, 0.75\}$ but is beaten by the latter in smaller instances. `ILP-cpws` is almost always behind both `freecp` and `LBBD-opt`, except when $\alpha = 1$ where `freecp` noticeably underperforms even `ILP-fsws`. However, for this value of α `LBBD-opt` has a clearest advantage over other methods.

Dense instances. In dense instances, `LBBD-opt` stays superior compared to the other methods, both in objective value (see Table 8) and in computation time (see Figure 8). `LBBD-opt` proves the most instances optimal at all α values (Figure 8) and delivers the best average gain over `ILP-fsws` at $\alpha \in \{0.25, 0.75, 1.0\}$ (averaging respectively 3.62%, 4.90% and 6.72% across sets), tied with `freecp` at $\alpha = 0.5$ (averaging 4.48% vs 4.52%), as shown in Table 8. As in the standard case, the decomposition's edge becomes more pronounced on the larger instances. In terms of objective value, `freecp` again improves on `ILP-cpws` for the smaller α (e.g. 3.43% versus 2.47% in $\alpha = 0.25$), but the ordering reverses at $\alpha = 1.00$, where `ILP-cpws` (3.76%) overtakes `freecp` (3.23%).

Number of cuts generated. Table 9 reports the average number of feasibility and optimality cuts added during the search by `LBBD-opt`. Interestingly, two regularities emerge. First, feasibility cuts are consistently rare, with

	freecp	ILP-cpws	LBBD-opt	freecp	ILP-cpws	LBBD-opt
Set		$\alpha = 0.25$			$\alpha = 0.5$	
1	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**
2	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**
3	2.90	2.93	**2.95**	**0.95**	0.93	0.89
4	6.35	5.79	**6.55**	**5.07**	4.83	4.91
5	6.55	6.44	**6.66**	**4.65**	4.62	4.63
6	8.27	7.71	**8.51**	**6.44**	6.09	6.35
7	6.53	6.28	**6.54**	**5.16**	4.84	4.94
8	**6.89**	6.61	6.71	**5.16**	4.39	5.01
9	4.83	6.09	**6.30**	**4.22**	3.98	4.16
10	**4.77**	3.46	4.76	**3.82**	2.72	3.30
11	**4.23**	3.16	4.19	**2.98**	1.68	2.94
12	**4.98**	4.21	4.97	3.42	2.48	**3.45**
13	**4.64**	3.45	4.62	**3.25**	2.54	3.24
14	4.78	5.08	**5.63**	4.27	3.35	**4.96**
15	4.66	4.30	**5.15**	4.29	3.48	**4.59**
Mean	*4.69*	*4.37*	***4.90***	***3.58***	*3.06*	*3.56*
		$\alpha = 0.75$			$\alpha = 1.0$	
1	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**
2	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**
3	0.47	**0.77**	0.69	**0.00**	**0.00**	**0.00**
4	2.57	2.54	**2.63**	-0.06	**0.00**	**0.00**
5	2.22	2.28	**2.67**	-0.38	**0.00**	**0.00**
6	3.69	3.27	**3.70**	-1.93	-0.00	**0.00**
7	**4.06**	2.72	3.99	-3.08	0.02	**0.02**
8	**15.32**	6.95	10.30	-4.17	0.16	**0.17**
9	3.17	2.96	**3.59**	-3.38	0.74	**0.74**
10	**3.28**	2.58	3.00	2.29	3.73	**5.77**
11	2.29	1.82	**2.67**	1.12	3.85	**5.28**
12	2.12	1.76	**2.37**	1.88	5.11	**6.53**
13	3.05	2.26	**3.91**	5.86	9.86	**11.71**
14	3.76	2.55	**4.35**	11.87	12.63	**17.85**
15	3.58	2.80	**3.69**	11.58	14.71	**17.30**
Mean	***3.30***	*2.35*	*3.17*	*1.44*	*3.39*	***4.36***

Table 7: Performance comparison relative to ILP-fsws across different α values on standard instance sets. The results are given as percentages. The best approaches per set is reported in bold.

on average between 1.47 and 3.72 per instance. It means that the decomposition only seldom eliminates a master solution because its subproblem is infeasible. Second, optimality cuts are more numerous but decrease steeply with α, from 72.51 at $\alpha = 0.25$ to 7.58 at $\alpha = 1.00$, mirroring the diminishing weight of the makespan in the objective and hence the diminishing amount of refinement the master needs. Therefore, few cuts are injected into the master, which is one reason for the strong performance of LBBD-opt, especially when α is close to 1. Finally, we defer the sparse sets to the next subsection: as detailed there, in the large majority of sparse instances the LBBD approach cannot certify the exactness of its solution, so they fall outside the optimality-guaranteed pool considered here.

6.4. Comparison LBBD-heur — ILP — freecp

In this section, let us reintroduce the instances in which the LBBD-heur approach may not guarantee optimality of the solution. Table 10 shows how unevenly this affects the three densities: optimality is certified on all dense instances and on between 96.9% and 99.7% of the standard ones, but on only 15% to 20% of the sparse instances when $\alpha \leq 0.75$. This behavior can be explained by the fact that in standard, and even more in dense ones, the constraints (5.15) significantly reduce the possibilities that the other tasks can take. Thus, in sparse instances, the *subproblem* cannot close the optimal gap before the time limit. It is hardly due to the *subproblem* struggling with its feasibility decidability. Indeed, the percentage of instances without an optimality guarantee plummets to zero when the *subproblem* is a feasibility problem, i.e., when $\alpha = 1$. Table 11 displays the average percentage of *subproblems* that do not prove optimality, per set, and α. As expected, the larger the instance, the more of them fail in it. Because so few standard and dense instances lack a guarantee, the behavior of LBBD-heur on those densities coincides with the LBBD-opt analysis of Section 6.3, to which we refer the reader.

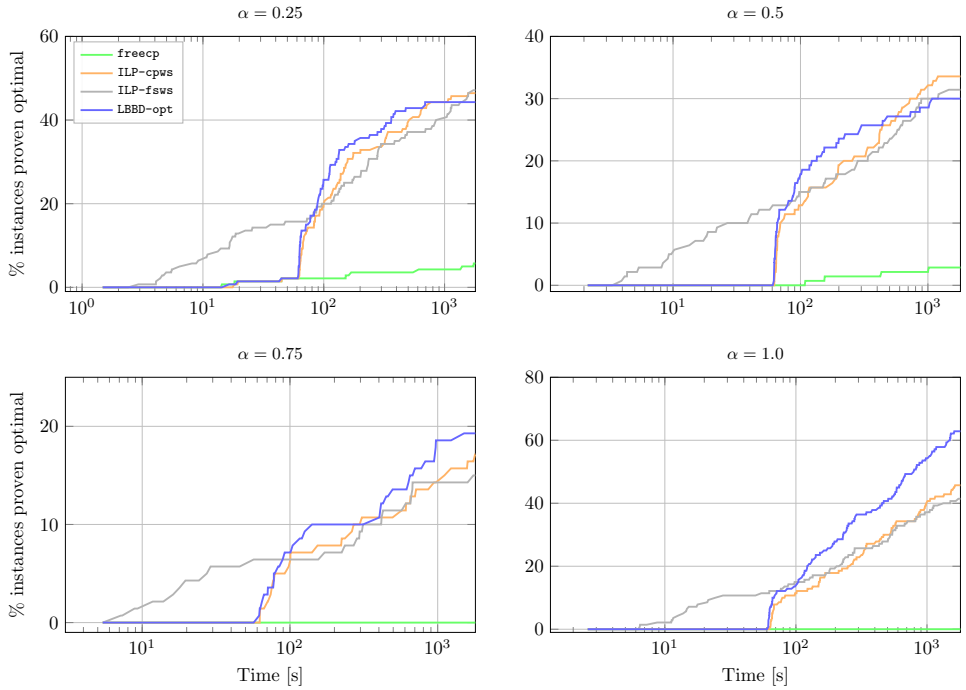


Figure 8: Comparison of the percentage of solutions solved to optimality over time for each method in dense sets.

Sparse instances. In sparse instances, `LBBD-heur` tends to be better than the other approaches in terms of objective value only when $\alpha = 1$ (12.46% gain over `ILP-fsws`, versus 9.72% for `freecp`), as shown in Table 12. In the other cases, the `freecp` approach is more efficient, reaching gains of 21.13%, 25.49%, 12.73% at $\alpha = 0.25, 0.50$ and 0.75, respectively. `LBBD-heur` is also the method that converges the fastest to a solution it considers "optimal" (which may not be the true global optimum), as shown in Figure 9.

Statistical analysis. To put these comparisons on a firmer footing, Table 13 reports Holm-corrected p-values from one-sided paired Wilcoxon signed-rank tests (Wilcoxon, 1945; Holm, 1979) comparing `LBBD-heur` to each other method on the per-instance objective. Against `ILP-cpws` and `ILP-fsws`, `LBBD-heur` is significantly better ($p < 0.001$) in all density, establishing uniform dominance over the ILP baselines throughout the experimental grid. The comparison against `freecp` is more nuanced. `LBBD-heur` is statistically better ($p < 0.001$) than `freecp` for all densities when $\alpha = 1$. In sparse and dense instances, the advantage of `LBBD-heur` tends to decrease when α lowers, so in those instances `LBBD-heur` and `freecp` are statistically comparable in objective value, and in sparse sets when $\alpha \leq 0.5$ `freecp` even achieves the higher mean gains (Table 12). In standard instances, the Wilcoxon test reveals that in the case $\alpha = 0.5$ `LBBD-heur` is not superior to `freecp`, as Table 7 also shows, but for the other values of α, it appears to be the best approach.

Taken together, these results identify `LBBD-heur` as the strongest method whenever a certified optimum is attainable and, at the objective value, wherever the makespan leaves the objective: its advantage over `freecp` is concentrated at $\alpha = 1.0$ in all densities (and also at $\alpha \in \{0.25, 0.75\}$ in standard sets), while for smaller α in the sparse instances the monolithic CP remains competitive and is frequently the better choice.

6.5. Discussion on results

In summary, the LBBD approach is the best approach by far, in terms of solution quality and computation time, when $\alpha = 1$, i.e., when *subproblem* collapses to a feasibility oracle. In this case, no matter the density of the instance, the CP modeling the subproblem always decides the feasibility before the time limit, meaning the optimal solution yielded is guaranteed to be the global optimum. Experiments show that LBBD approach can solve to optimality instances up to 480 tasks whose around 86 of them are energy-intensive, and a time horizon of up to 6207. It remains a relevant approach in dense and standard instances when $\alpha < 1$, in which it nearly often guarantees the validity of its optimal solution, and beats the other approaches in terms of computation

	freecp	ILP-cpws	LBBD-opt	freecp	ILP-cpws	LBBD-opt
Set		$\alpha = 0.25$			$\alpha = 0.5$	
1	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**
2	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**
3	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**
4	**1.64**	1.39	0.86	2.86	**3.40**	2.34
5	7.29	5.60	**10.41**	12.65	6.69	**12.82**
6	7.83	5.92	**8.56**	8.32	5.11	**9.64**
7	**7.22**	4.36	5.51	**7.81**	4.73	6.59
Mean	*3.43*	*2.47*	***3.62***	***4.52***	*2.85*	*4.48*
		$\alpha = 0.75$			$\alpha = 1.0$	
1	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**
2	**0.02**	0.00	-0.01	-1.72	**0.00**	**0.00**
3	**1.37**	1.26	1.15	-3.57	0.25	**0.25**
4	**3.42**	3.38	3.35	4.17	8.24	**10.09**
5	9.30	7.25	**12.63**	5.95	6.99	**11.73**
6	8.87	4.85	**11.11**	8.73	5.37	**15.32**
7	**8.87**	4.61	6.10	9.08	5.44	**9.63**
Mean	*4.55*	*3.05*	***4.90***	*3.23*	*3.76*	***6.72***

Table 8: Performance comparison relative to ILP-fsws in dense instances across different α values. The results are given as percentages. The best approach per set is reported in bold.

α	Avg optimality cuts	Avg feasibility cuts
0.25	72.51	2.75
0.50	63.44	3.72
0.75	16.31	1.47
1.00	7.58‡	1.68‡

Table 9: Average number of LBBD-opt optimality and feasibility cuts in the standard instances. ‡: for $\alpha = 1$, no cut is added; instead, the optimality cuts represent the number of subproblems in which a feasible solution was found.

α	% guaranteed optimal solutions		
	Sparse	Standard	Dense
0.25	15.0	96.90	100.0
0.50	19.0	97.25	100.0
0.75	20.0	98.98	100.0
1.00	100.0	99.67	100.0

Table 10: Percentage of solutions guaranteed optimal across instance densities under a 20s *subproblem* time limit. A solution is guaranteed optimal if none of its *subproblems* reached the time limit.

speed and solution quality, with an exception for $\alpha = 0.5$ in standard instances and $\alpha = 0.25$ in dense instances, in which freecp matches the LBBD. Finally, in sparse instances, freecp seems to be the best approaches when $\alpha < 1$, because most of the LBBD subproblems timeout, hampering significantly the exploration of search space, potentially pruning optimal solutions while considerably slowing down the master search tree exploration.

7. Generalization to different resource environments

In this section, we examine the applicability of the proposed decomposition scheme to additional energy-aware optimization problems that arise in heterogeneous machine environments. Specifically, we demonstrate how our method can be adapted to two further scheduling problems: the Resource-Constrained Project Scheduling Problem (RCPSP) with blocking times under a total weighted tardiness objective, and the Flexible Job Shop Scheduling Problem. In both cases, scheduling problems are integrated with a time-of-use electricity tariff structure and machine state modeling. We then apply the proposed decomposition framework to each of these combined problem settings. In the following, we detail only the subproblems, since the master problem is technically unchanged. Hence, the notations defined in Section 3 still hold. The time limit for the subproblem is still set at 20 seconds.

7.1. Subproblem: RCPSP with blocking times & total weighted tardiness

In this section, we investigate another variant of the RCPSP, $PSm|intree|\sum_j w_j T_j$, detailed in Nedbálek and Novák (2025), which serves as the subproblem. In it, each task $j \in \mathcal{T}$ has a processing duration p_j and a due date d_j. We define the tardiness weight w_j as the penalty for each task j that is late in a given period of time. The precedence constraints between tasks are represented by a graph $G = (V, A)$. Task preemption is not permitted. Tasks are assigned to resources $\mathcal{R} = \{R_0, \ldots, R_m\}$ with time-variant renewable capacities, i.e.,

α	Set																		
	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
0.25	4.5	58.9	59.8	80.8	90.5	71.8	66.3	93.6	78.5	95.9	92.5	95.5	88.5	95.3	95.3	94.5	88.3	88.6	86.7
0.50	-	68.9	38.3	78.0	72.3	59.9	55.0	81.4	70.9	79.9	94.7	93.3	93.6	83.5	75.9	84.2	87.2	78.6	78.3
0.75	12.5	64.5	36.5	63.6	69.6	30.1	39.6	49.4	62.3	58.1	42.1	84.6	74.9	72.4	61.4	69.1	78.8	66.5	80.3

Table 11: Share of `LBBD-heur` subproblems which reached the 20s-subproblem time limit per sparse instance set and per α value, restricted to sparse instances for which at least one subproblem has timed out.

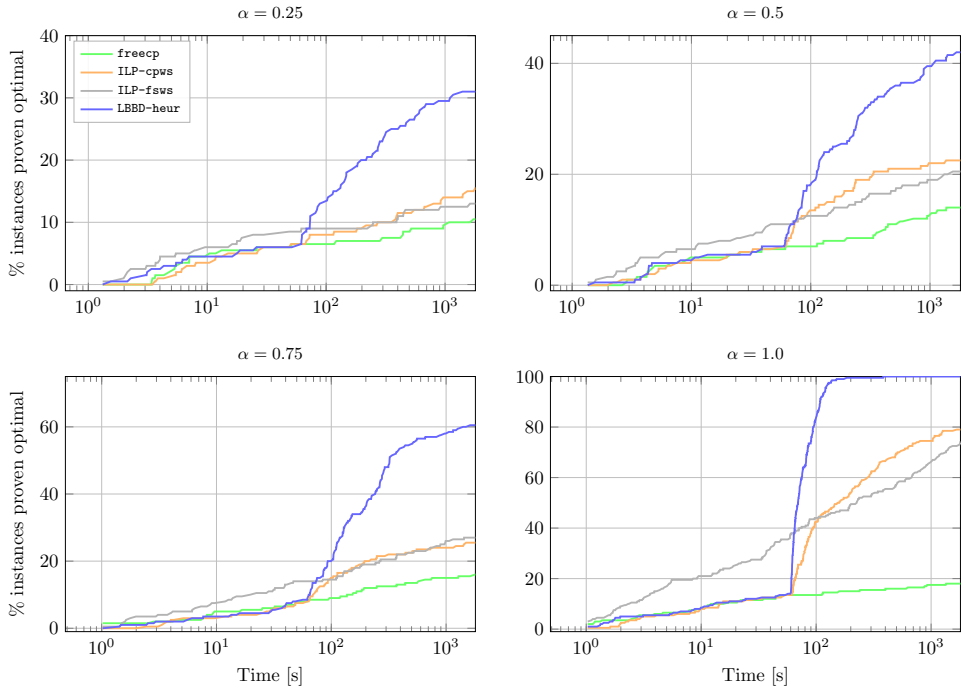


Figure 9: Comparison of the percentage of solutions solved to optimality‡ over time for each method in sparse sets. ‡: Note that, in the case of `LBBD-heur`, this implies that the solver certifies the optimality of the incumbent solution; however, due to the inefficacy of the cuts generated in the *subproblem*, this solution may in fact not correspond to the true global optimum.

at any time $t \in \mathcal{I}$, the capacity of a resource k is $\rho_k^{(t)}$. We assume that $\rho_k^{(t)} \in \{0, r_k\}$ with $r_k > 0$, except for the energy-aware resource, which has a constant capacity of 1 at any time. At any time $t \in \mathcal{I}$, a task j would consume a quantity $r_{jk}^{(t)}$ of resource k if processed. The subproblem can be modeled by the following constraint programming formulation:

$$(\textit{Subproblem}): \quad \min \quad \sum_{j \in \mathcal{T}} w_j \max\{\textsc{EndOf}(\xi_j) - d_j, 0\} \tag{7.1}$$

$$s.t. \quad \sum_{j \in \mathcal{T}} \textsc{Pulse}\left(\xi_j, r_{jk}^{(t)}\right) \leq \rho_k^{(t)} \qquad \forall t \in \mathcal{I}, k \in \mathcal{R} \tag{7.2}$$

$$\textsc{EndBeforeStart}(\xi_i, \xi_j) \qquad \forall (i,j) \in \mathcal{T}^2,\ i \prec j \tag{7.3}$$

$$\textsc{StartOf}(\xi_j) = t \qquad \forall j \in \mathcal{J}, t \in \mathcal{I}, \hat{x}_{jt} = 1 \tag{7.4}$$

$$\textsc{Interval}\ \xi_j,\ \textsc{LengthOf}(\xi_j) = p_j \qquad \forall j \in \mathcal{T} \tag{7.5}$$

We compare our cold-started LBBD approach with a monolithic CP formulation derived from the CP proposed in Section 4 on a small set of instances derived from the dataset[3] proposed by Nedbálek and Novák (2025). For each instance, we randomly select 20% of the tasks to be energy-intensive. Table 14 presents, for different values of α, the minimal, maximal, and average gaps (E.1) of the LBBD approach relative to the CP over the data set and the associated standard deviations. It reveals that our approach exhibits superior

[3]The dataset can be retrieved from [dataset] Juvigny (2025c)

	freecp	ILP-cpws	LBBD-heur	freecp	ILP-cpws	LBBD-heur
Set	$\alpha = 0.25$			$\alpha = 0.5$		
1	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**
2	**0.32**	0.12	**0.32**	**0.00**	**0.00**	**0.00**
3	0.97	0.55	**1.18**	**0.39**	**0.00**	0.35
4	4.29	3.85	**4.55**	**0.83**	0.57	0.81
5	**3.79**	2.71	3.75	3.03	2.61	**3.44**
6	**8.46**	5.60	8.33	5.36	3.74	**6.03**
7	10.99	10.45	**11.67**	7.60	6.34	**8.28**
8	18.94	19.27	**20.85**	10.78	16.03	**17.22**
9	27.45	27.68	**29.40**	29.00	**32.73**	29.59
10	**22.87**	21.11	22.64	**25.90**	25.38	15.73
11	**25.52**	23.68	24.91	26.86	25.52	**26.99**
12	**23.07**	12.50	20.18	**24.08**	23.85	22.62
13	**35.58**	27.93	30.58	**47.13**	34.91	36.90
14	**39.45**	31.97	33.92	**56.63**	41.71	44.84
15	31.91	27.47	**34.45**	**41.04**	33.97	29.76
16	**33.15**	19.83	27.61	**45.66**	22.26	32.82
17	**40.40**	35.37	35.85	**53.35**	44.11	36.18
18	**29.08**	15.84	17.76	**45.48**	29.48	26.45
19	**37.63**	21.08	25.38	**44.26**	25.38	27.48
20	**28.79**	21.05	27.94	**42.36**	32.99	41.18
Mean	***21.13***	*16.40*	*19.06*	***25.49***	*20.08*	*20.33*
	$\alpha = 0.75$			$\alpha = 1.0$		
1	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**
2	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**
3	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**	**0.00**
4	**0.58**	0.45	0.53	**0.00**	**0.00**	**0.00**
5	-0.26	0.03	**0.69**	**0.00**	**0.00**	**0.00**
6	**4.17**	3.22	4.15	**0.00**	**0.00**	**0.00**
7	0.13	0.64	**1.14**	**0.00**	**0.00**	**0.00**
8	-0.15	**0.64**	0.04	**0.00**	**0.00**	**0.00**
9	-29.03	-30.91	**-28.59**	-1.12	**0.00**	**0.00**
10	**21.88**	19.41	-1.68	-2.83	**0.00**	**0.00**
11	26.40	24.68	**26.63**	-0.28	**0.00**	**0.00**
12	**15.81**	12.66	-12.15	-0.01	**0.00**	**0.00**
13	22.36	**43.44**	33.71	3.07	13.42	**13.42**
14	**29.10**	24.54	28.25	13.68	0.55	**19.22**
15	25.38	31.13	**46.80**	33.49	28.31	**36.96**
16	**28.07**	15.64	28.07	9.83	-4.80	**15.71**
17	**29.31**	18.71	18.99	30.34	28.54	**34.36**
18	**15.80**	13.74	14.62	34.10	33.34	**44.05**
19	37.85	31.20	**39.30**	46.91	41.49	**53.46**
20	27.16	31.14	**35.01**	27.16	27.75	**32.11**
Mean	***12.73***	*12.02*	*11.78*	*9.72*	*8.43*	***12.46***

Table 12: Performance comparison relative to ILP-fsws in sparse instances across different α values. The results are given as percentages. The best approach per set is reported in bold.

performance in terms of identifying optimal solutions compared to the CP for all instances, with an average improvement of 7.43%, with a maximum reaching nearly 50%. It is also worth noting that the LBBD never performs worse than the CP. Also, no subproblems reached the time limit, there were all solved to optimality. Furthermore, Figure 10 depicts how quickly our LBBD approach proves the optimality of solutions, with nearly 90% of instances solved to optimality in less than half an hour, whereas the CP could only prove the optimality of less than 5% of them in the same time.

7.2. Subproblem: Flexible job shop

In this subsection, we consider a classical flexible job shop problem (FJSP) $FJm||C_{\max}$ (Dauzère-Pérès et al., 2024). Let $\mathcal{M}$ be a set of m machines. Each machine cannot handle more than one job at a time. We have a set of n jobs $\mathfrak{J}$. Each job $i \in \mathfrak{J}$ comprises a chain $\mathcal{O}_i$ of operations $o_{i\lambda}$, so the set of all operations is $\mathcal{T} = \bigcup_{i\in\mathfrak{J}} \mathcal{O}_i$. Each operation i can be processed by any machine in a subset of machines $\mathcal{M}_i \subseteq \mathcal{M}$. We use the MILP detailed in Dauzère-Pérès et al. (2024). Let us consider binary variables $\alpha_i^k = 1$ when operation i starts on machine k and $\beta_{ij} = 1$ when operation i is sequenced before j, and continuous variables $C_{\max}$ and t_i, the MILP subproblem is:

Density	α	`freecp`	`ILP-cpws`	`ILP-fsws`
Sparse	0.25	*1.000*	<0.001	<0.001
	0.50	*0.896*	<0.001	<0.001
	0.75	*0.341*	<0.001	<0.001
	1.00	<0.001	<0.001	<0.001
Standard	0.25	0.005	<0.001	<0.001
	0.50	*0.998*	<0.001	<0.001
	0.75	0.067	<0.001	<0.001
	1.00	<0.001	<0.001	<0.001
Dense	0.25	*0.770*	<0.001	<0.001
	0.50	*0.130*	<0.001	<0.001
	0.75	*0.131*	<0.001	<0.001
	1.00	<0.001	<0.001	<0.001

Table 13: Pairwise comparison of `LBBD-heur` against each of the three competitors on the per-instance objective. Each cell reports the p-value of a one-sided paired Wilcoxon signed-rank test (Wilcoxon, 1945), the pairing being by instance: instances on which the two methods returned the same objective are dropped, the remaining differences are ranked by magnitude, and the alternative is H_1: `LBBD-heur` produces a strictly smaller (better) objective than the competitor. Within each density $\times$ α row, the three resulting p-values are then Holm-corrected (Holm, 1979) to control the family-wise error rate across the three competitors. A value $p = 0.1$ therefore indicates that, after multiplicity correction, `LBBD-heur` is significantly better than the corresponding competitor at the 10 % level on the underlying instance pool. The values for which no significant statistical correlation is found are reported in italic.

	Relative to CP [%]			
α	mean	min	max	std
0.25	8.09	0.00	36.00	10.67
0.50	6.64	0.00	35.93	9.42
0.75	7.80	0.00	48.70	11.61
1.00	7.17	0.00	30.81	9.02

Table 14: RCPSP with blocking times & total weighted tardiness - Performance comparison for various values of α of the LBBD approach relative to the monolithic CP presented in Section 7.1. A time limit of 1800 seconds is imposed for both methods. std = standard deviation.

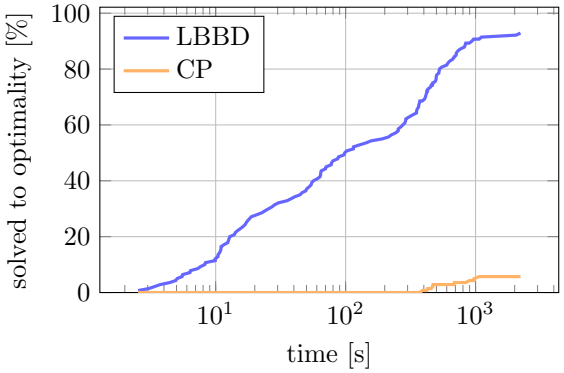


Figure 10: RCPSP with blocking times & total weighted tardiness - Proportion of instances solved to optimality by each method over time. The results for each value of $\alpha \in \{0.25, 0.5, 0.75, 1.0\}$ are concatenated.

$$(\textit{Subproblem}): \ \min C_{\max} \tag{7.6}$$

$$s.t. \sum_{k \in \mathcal{M}_i} \alpha_i^k = 1 \qquad \forall i \in \mathcal{T} \tag{7.7}$$

$$t_i \geq t_{pr(i)} + \sum_{k \in \mathcal{M}_{pr(i)}} p_{pr(i)}^m \alpha_{pr(i)}^k \qquad \forall i \in \mathcal{T} \tag{7.8}$$

$$t_i \geq t_j + p_j^k - M\left(2 - \alpha_i^k - \alpha_j^k + \beta_{ij}\right) \qquad \forall (i,j) \in \mathcal{T}^2, \text{ if } i \neq j, k \in \mathcal{M}_i \cap \mathcal{M}_j \tag{7.9}$$

$$t_j \geq t_i + p_i^k - M\left(3 - \alpha_i^k - \alpha_j^k - \beta_{ij}\right) \qquad \forall (i,j) \in \mathcal{T}^2, \text{ if } i \neq j, k \in \mathcal{M}_i \cap \mathcal{M}_j \tag{7.10}$$

$$C_{\max} \geq t_i + \sum_{k \in \mathcal{M}_i} p_i^k \alpha_i^k \qquad \forall i \in \mathcal{T} \tag{7.11}$$

$$t_j = t \qquad \forall j \in \mathcal{J}, t \in \mathcal{I}, \hat{x}_{jt} = 1 \tag{7.12}$$

The objective function of the master is normalized as in (3.4). We compare LBBD with a monolithic MILP (see Appendix G of the supplementary materials) based on the time-indexed formulation of FJSP presented by Ku and Beck (2016). It is interesting to notice that the SPACES graph forces the monolithic ILP to follow a time-indexed formulation, while the subproblem of our LBBD can employ models that do not have to stick to this kind of formulation; in particular, the one we use in the subproblem does not. No warmstart solution is

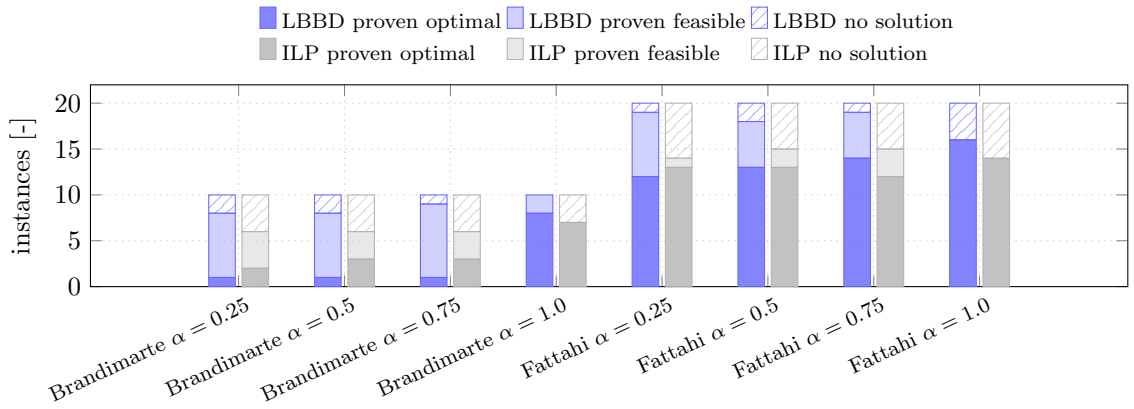


Figure 11: Flexible Job Shop - Number of solutions with proven optimality and feasible solution found grouped by benchmark set and α value.

supplied to models. We employ the IIS (Irreducible Inconsistent Subsystem) (Gleeson and Ryan, 1990) provided by the solver to determine $\mathcal{I}nf$. The tests are performed on a dataset[4] created from the Brandimarte (1993) and Fattahi and Fallahi (2010) datasets, in which around 20% of the operations are labeled energy-intensive. Figure 11 shows the number of instances in which LBBD and ILP have found an optimal solution for each dataset and each value of α. We observe that ILP is better to prove optimality when α is low, while LBBD is always better when $\alpha = 1$. Although the methods' performances remain close, ILP seems slightly better at proving the optimality of a solution in the other cases, especially in the Brandimarte set. However, a general trend tends to emerge when the size of the instances grows, in which the LBBD digs out a feasible solution more often than the monolithic MILP. Indeed, as shown in Figure 11, in the 120 overall experiments run, with a time limit of 1800 seconds, the LBBD returned a feasible solution 107 times, while the monolithic ILP was only able to find one 83 times, i.e., around 20% less.

7.3. Additional remarks

In the problems mentioned in Sections 7.1 and 7.2, our LBBD approach preserves the optimality of the final solution, as long as the subproblem is solved to optimality each time it is triggered. However, if this optimality criterion is not mandatory, approximation methods can be used to compute the subproblem. In particular, a wide variety of heuristics or metaheuristic approaches can be envisioned. They seem to be particularly suited for cases where the subproblem would be very complex but only checks the feasibility of the master solution.

8. Conclusion

In this work, we have investigated the classical resource-constrained project scheduling problem extended with time-of-use energy tariffs and machine states. We proposed three models for this problem: two monolithic models employing *ILP* and *CP* approaches, and *LBBD*. In the latter, the problem was divided into two problems: a master problem, solved by an ILP, solves the energy part of the original problem, while the RCPSP components are put aside in a subproblem. This subproblem is solved efficiently via a CP model. We assessed these models in a series of energy-intensive density instances. Outcomes revealed that in standard and dense instances, LBBD outperforms the other proposed approaches, with increasing margins when the instance size grows, solving to optimality instances with up to 480 tasks. In sparse instances, it is the monolithic CP that tends to yield the best results, as long as the makespan appears in the objective function. Otherwise, LBBD statistically retains its dominance in most cases. Finally, we have demonstrated that LBBD is applicable to a broader class of problems, provided that they admit a decomposition into an energy management problem in which the start times of energy-intensive activities can be determined within the subproblem. We investigated two distinct

[4]The dataset can be retrieved from the [dataset] Juvigny (2025b)

subproblem formulations: a resource-constrained project scheduling problem (RCPSP) with blocking times and a total weighted tardiness objective, and a classical flexible job shop scheduling problem. In both cases, `LBBD` outperforms a traditional monolithic solution approach.

In further work, we can envision having multiple machines that require energy-intensive resources. These machines may operate on the same energy source or require different energy-constrained resources, each with its own TOU. Furthermore, we can also consider more than three states, but with multiple idle/processing times, as some tasks require a precise state to be performed. We would also suggest looking into the impact of temporary energy storage capacities, such as batteries, especially when combined with on-site energy production capacities (i.e., independent of time-of-use), such as solar panels or wind turbines. Finally, another avenue of research would be to explore the combined effect of uncertainty of energy tariffs (Xiechen et al., 2026) and state switching over longer time horizons.

Acknowledgments

This work was co-funded by the European Union under the project ROBOPROX (reg. no. CZ.02.01.01/00/22_008/0004590) and by the Grant Agency of the Czech Republic under the Project GACR 25-17904S.

References

Aghelinejad, M., Ouazene, Y., and Yalaoui, A. (2018). Production scheduling optimisation with machine state and time-dependent energy costs. *International Journal of Production Research*, 56(16):5558–5575.

Aghelinejad, M., Ouazene, Y., and Yalaoui, A. (2019). Complexity analysis of energy-efficient single machine scheduling problems. *Operations Research Perspectives*, 6:100105.

Artigues, C., Demassey, S., and Neron, E. (2013). *Resource-constrained project scheduling: models, algorithms, extensions and applications*. John Wiley & Sons.

Artigues, C., Hartmann, S., and Vanhoucke, M. (2025). Fifty years of research on resource-constrained project scheduling explored from different perspectives. *European Journal of Operational Research*.

Artigues, C., Michelon, P., and Reusser, S. (2003). Insertion techniques for static and dynamic resource-constrained project scheduling. *European Journal of Operational Research*, 149(2):249–267.

Benders, J. (1962). Partitioning procedures for solving mixed-variables programming problems. *Numer. Math*, 4(1):238–252.

Benedikt, O., Módos, I., and Hanzálek, Z. (2020). Power of pre-processing: production scheduling with variable energy pricing and power-saving states. *Constraints*, 25(3):300–318.

Benedikt, O., Módos, I., Novak, A., and Hanzálek, Z. (2026). Green scheduling with time-of-use tariffs and machine states: Optimizing energy cost via branch-and-bound and bin packing strategies. *European Journal of Operational Research*, 328(1):64–77.

Berthold, T., Heinz, S., Lübbecke, M. E., Möhring, R. H., and Schulz, J. (2010). A constraint integer programming approach for resource-constrained project scheduling. In *International Conference on Integration of Artificial Intelligence (AI) and Operations Research (OR) Techniques in Constraint Programming*, pages 313–317. Springer.

Blazewicz, J., Lenstra, J. K., and Kan, A. R. (1983). Scheduling subject to resource constraints: classification and complexity. *Discrete applied mathematics*, 5(1):11–24.

Bokde, N. D., Pedersen, T. T., and Andresen, G. B. (2021). Optimal scheduling of flexible power-to-x technologies in the day-ahead electricity market. *arXiv preprint arXiv:2110.09800*.

Brandimarte, P. (1993). Routing and scheduling in a flexible job shop by tabu search. *Annals of Operations research*, 41(3):157–183.

Chakrabortty, R. K., Sarker, R., and Essam, D. (2015). Resource constrained project scheduling: A branch and cut approach. In *Proceedings of the 45th international conference on computers and industrial engineering Metz*, volume 132.

Chen, B. and Zhang, X. (2019). Scheduling with time-of-use costs. *European Journal of Operational Research*, 274(3):900–908.

[dataset] Juvigny, C. (2025a). Dense instances for RCPSP with time-of-use tariffs and machine states, https://doi.org/10.7910/DVN/SH2DSA.

[dataset] Juvigny, C. (2025b). Instances for FJSP with time-of-use tariffs and machine states, https://doi.org/10.7910/DVN/AXMPOS.

[dataset] Juvigny, C. (2025c). Instances for RCPSP with blocking times & machine states & TOU & total weighted tardiness criteria, https://doi.org/10.7910/DVN/XI9JE0.

[dataset] Juvigny, C. (2025d). Instances of standard density for RCPSP with time-of-use tariffs and machine states, https://doi.org/10.7910/DVN/EPX98Z.

[dataset] Juvigny, C. (2025e). Sparse instances for RCPSP with time-of-use tariffs and machine states, https://doi.org/10.7910/DVN/MQS0NY.

Dauzère-Pérès, S., Ding, J., Shen, L., and Tamssaouet, K. (2024). The flexible job shop scheduling problem: A review. *European Journal of Operational Research*, 314(2):409–432.

Ding, J.-Y., Song, S., Zhang, R., Chiong, R., and Wu, C. (2015). Parallel machine scheduling under time-of-use electricity prices: New models and optimization approaches. *IEEE Transactions on Automation Science and Engineering*, 13(2):1138–1154.

Du, B., Tan, T., Guo, J., Li, Y., and Guo, S. (2021). Energy-cost-aware resource-constrained project scheduling for complex product system with activity splitting and recombining. *Expert Systems with Applications*, 173:114754.

Fang, K., Uhan, N., Zhao, F., and Sutherland, J. W. (2011). A new approach to scheduling in manufacturing for power consumption and carbon footprint reduction. *Journal of Manufacturing Systems*, 30(4):234–240.

Fang, K., Uhan, N. A., Zhao, F., and Sutherland, J. W. (2016). Scheduling on a single machine under time-of-use electricity tariffs. *Annals of Operations Research*, 238(1):199–227.

Fattahi, P. and Fallahi, A. (2010). Dynamic scheduling in flexible job shop systems by considering simultaneously efficiency and stability. *CIRP Journal of Manufacturing Science and Technology*, 2(2):114–123.

Fazli Khalaf, A. and Wang, Y. (2018). Energy-cost-aware flow shop scheduling considering intermittent renewables, energy storage, and real-time electricity pricing. *International Journal of Energy Research*, 42(12):3928–3942.

Gaggero, M., Paolucci, M., and Ronco, R. (2023). Exact and heuristic solution approaches for energy-efficient identical parallel machine scheduling with time-of-use costs. *European Journal of Operational Research*, 311(3):845–866.

Gahm, C., Denz, F., Dirr, M., and Tuma, A. (2016). Energy-efficient scheduling in manufacturing companies: A review and research framework. *European Journal of Operational Research*, 248(3):744–757.

Geng, K., Ye, C., Dai, Z. H., and Liu, L. (2020). Bi-objective re-entrant hybrid flow shop scheduling considering energy consumption cost under time-of-use electricity tariffs. *Complexity*, 2020(1):8565921.

Gleeson, J. and Ryan, J. (1990). Identifying minimally infeasible subsystems of inequalities. *ORSA Journal on Computing*, 2(1):61–63.

Graham, R., Lawler, E., Lenstra, J., and Kan, A. (1979). Optimization and approximation in deterministic sequencing and scheduling: a survey. In Hammer, P., Johnson, E., and Korte, B., editors, *Discrete Optimization II*, volume 5 of *Annals of Discrete Mathematics*, pages 287–326. Elsevier.

Hall, N. G. and Posner, M. E. (2026). Optimal energy scheduling under time-of-use tariffs. *Journal of Scheduling*, 29(2):157–171.

Heinz, V., Novák, A., Vlk, M., and Hanzálek, Z. (2022). Constraint programming and constructive heuristics for parallel machine scheduling with sequence-dependent setups and common servers. *Computers & Industrial Engineering*, 172:108586.

Heinz, V., Vilím, P., and Hanzálek, Z. (2025). Reinforcement learning for search tree size minimization in constraint programming: New results on scheduling benchmarks. *Computers & Industrial Engineering*, 209:111413.

Herroelen, W., De Reyck, B., and Demeulemeester, E. (1998). Resource-constrained project scheduling: A survey of recent developments. *Computers & Operations Research*, 25(4):279–302.

Ho, M. H., Hnaien, F., and Dugardin, F. (2022). Exact method to optimize the total electricity cost in two-machine permutation flow shop scheduling problem under time-of-use tariff. *Computers & Operations Research*, 144:105788.

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian journal of statistics*, pages 65–70.

Hooker, J. N. (2007). Planning and scheduling by logic-based benders decomposition. *Operations research*, 55(3):588–602.

Hooker, J. N. and Ottosson, G. (2003). Logic-based benders decomposition. *Mathematical Programming*, 96(1):33–60.

Hu, R., Li, Y.-L., Qian, B., Zhang, C.-S., and Liu, J. (2024). Logic-based Benders decomposition for bi-objective parallel machine selection and job scheduling with release dates and resource consumption. *Computers & Operations Research*, 164:106528.

Hung, Y.-C. and Michailidis, G. (2018). Modeling and optimization of time-of-use electricity pricing systems. *IEEE transactions on smart grid*, 10(4):4116–4127.

Jain, V. and Grossmann, I. E. (2001). Algorithms for hybrid milp/cp models for a class of optimization problems. *INFORMS Journal on computing*, 13(4):258–276.

Johnson, D. B. (1977). Efficient algorithms for shortest paths in sparse networks. *Journal of the ACM (JACM)*, 24(1):1–13.

Ku, W.-Y. and Beck, J. C. (2016). Mixed integer programming models for job shop scheduling: A computational analysis. *Computers & Operations Research*, 73:165–173.

Kurniawan, B., Song, W., Weng, W., and Fujimura, S. (2021). Distributed-elite local search based on a genetic algorithm for bi-objective job-shop scheduling under time-of-use tariffs. *Evolutionary Intelligence*, 14(4):1581–1595.

Laborie, P., Rogerie, J., Shaw, P., and Vilím, P. (2018). Ibm ilog cp optimizer for scheduling: 20+ years of scheduling with constraints at ibm/ilog. *Constraints*, 23(2):210–250.

Lei, D., Yuan, Y., Cai, J., and Bai, D. (2022). Decomposition approaches for parallel machine scheduling of step-deteriorating jobs to minimize total tardiness and energy consumption. *Complex & Intelligent Systems*, 8:1407–1422.

López-Ibáñez, M., Dubois-Lacoste, J., Pérez Cáceres, L., Birattari, M., and Stützle, T. (2016). The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43–58.

Maghsoudlou, H., Afshar-Nadjafi, B., and Niaki, S. T. A. (2021). A framework for preemptive multi-skilled project scheduling problem with time-of-use energy tariffs. *Energy Systems*, 12(2):431–458.

Möhring, R. H., Schulz, A. S., Stork, F., and Uetz, M. (2003). Solving project scheduling problems by minimum cut computations. *Management science*, 49(3):330–350.

Munlin, M. (2018). Solving resource-constrained project scheduling problem using metaheuristic algorithm. In *2018 5th International Conference on Electrical and Electronic Engineering (ICEEE)*, pages 344–349. IEEE.

Módos, I., Šůcha, P., and Hanzálek, Z. (2016). Robust scheduling for manufacturing with energy consumption limits. In *2016 IEEE 21st international conference on emerging technologies and factory automation (ETFA)*, pages 1–8. IEEE.

Naderi, B. and Roshanaei, V. (2022). Critical-path-search logic-based benders decomposition approaches for flexible job shop

scheduling. *INFORMS Journal on Optimization*, 4(1):1–28.

Nedbálek, L. and Novák, A. (2025). Bottleneck identification in resource-constrained project scheduling via constraint relaxation. In *Proceedings of the 14th International Conference on Operations Research and Enterprise Systems - ICORES*, pages 340–347. INSTICC, SciTePress.

Park, M.-J. and Ham, A. (2022). Energy-aware flexible job shop scheduling under time-of-use pricing. *International Journal of Production Economics*, 248:108507.

Pellerin, R., Perrier, N., and Berthaut, F. (2020). A survey of hybrid metaheuristics for the resource-constrained project scheduling problem. *European journal of operational research*, 280(2):395–416.

Pouramin, M., Mirzazadeh, A., Davari-Ardakani, H., Mosadegh, H., and Alajegerdi, E. (2024). Multi-mode resource-constrained project scheduling problem along with material ordering under time-of-use electricity tariffs and carbon taxes. *Annals of Operations Research*, pages 1–45.

Rocholl, J., Mönch, L., and Fowler, J. (2020). Bi-criteria parallel batch machine scheduling to minimize total weighted tardiness and electricity cost. *Journal of Business Economics*, 90(9):1345–1381.

Schulz, S., Buscher, U., and Shen, L. (2020). Multi-objective hybrid flow shop scheduling with variable discrete production speed levels and time-of-use energy prices. *Journal of Business Economics*, 90(9):1315–1343.

Seman, L. O., Rigo, C. A., Camponogara, E., and Bezerra, E. A. (2024). Benders decomposition for the energy aware task scheduling of constellations of nanosatellites. *Computers & Operations Research*, 165:106618.

Shrouf, F., Ordieres-Meré, J., García-Sánchez, A., and Ortega-Mier, M. (2014). Optimizing the production scheduling of a single machine to minimize total energy consumption costs. *Journal of Cleaner Production*, 67:197–207.

Siek, J. G., Lee, L.-Q., and Lumsdaine, A. (2001). *The Boost Graph Library: User Guide and Reference Manual*. Pearson Education.

Sprecher, A. and Kolisch, R. (1996). Psplib-a project scheduling problem library. *European Journal of Operational Research*, 96(1):205–216.

Wiest, J. D. (1967). A heuristic model for scheduling large projects with limited resources. *Management Science*, 13(6):B–359.

Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics bulletin*, 1(6):80–83.

Xiechen, Z., Eric, A., Feng, C., and Damien, R. (2026). A stochastic bi-objective unrelated parallel machine scheduling under time-of-use tariffs. *Computers & Operations Research*, page 107567.

Yang, S., Wang, J.-Q., and Xu, Z. (2024). Logic-based Benders decomposition for order acceptance and scheduling on heterogeneous factories with carbon caps. *Computers & Operations Research*, 168:106706.

Zhao, S., Zhou, H., Chu, F., Wang, D., and Gao, K. (2025). A twin-reinforced evolutionary algorithm for flexible job shop scheduling problem under time-of-use tariffs. *Computers & Industrial Engineering*, page 111754.

Zheng, H.-y. and Wang, L. (2015). Reduction of carbon emissions and project makespan by a pareto-based estimation of distribution algorithm. *International Journal of Production Economics*, 164:421–432.

Zuccato, F., Rodler, P., Friedrich, G., Schekotihin, K., and Comploi-Taupe, R. (2025). Energy-aware double-flexible job shop scheduling with machine modes and setup times: A real-world industrial case study using constraint programming. *ECAI Workshop on AI-based Planning for Complex Real-World Applications*.

Appendix A. Acronyms table

RCPSP	Resource-Constrained Project Scheduling Problem
(M)ILP	(Mixed) Integer Linear Programming
CP	Constraint Programming
LBBD	Logic-Based Benders Decomposition
TOU	Time-of-use tariffs
TEC	Total Energy Cost
LB/UB	Lower bounds / Upper bounds

Table A.15: Acronyms used throughout the paper.

Appendix B. SPACES graph

The SPACES graph, depicted in Figure B.12, is a time–state graph whose nodes are (state, interval) pairs of the energy-intensive machine over its state set Σ, and whose arcs weight each admissible state change by its transition time T and transition energy P at the prevailing time-of-use tariffs. The minimum-cost machine-state trajectory between any two proc intervals — and between the initial/final off state and any proc state — is obtained as a shortest path in this graph. SPACES therefore decouples the scheduling decisions from the optimization of the state trajectory: once the task start times are fixed, the optimal states filling the surrounding gaps follow directly from precomputed shortest paths, which sharply reduces the model size.

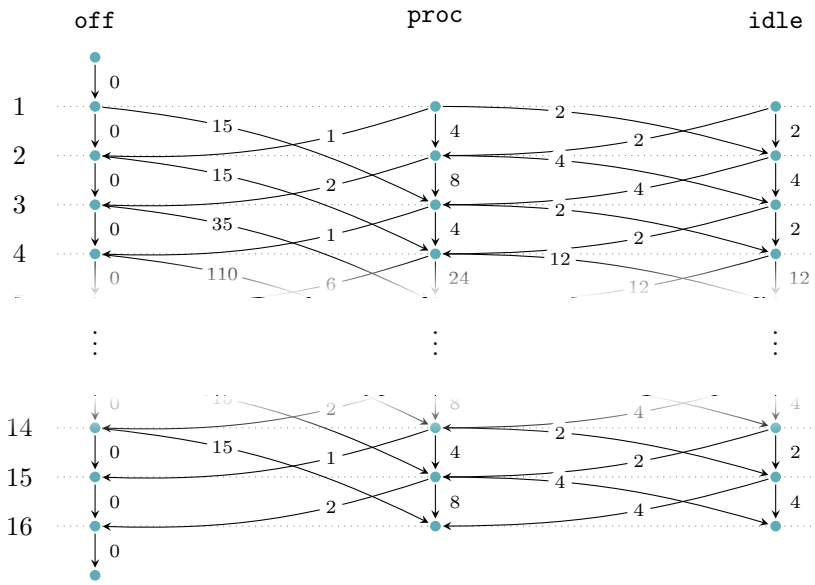


Figure B.12: SPACES graph for the transition graph in Figure 2 and energy cost profile from Figure 4.

Appendix C. Detailed characteristics of test instance sets

The following tables detail the characteristics of the instance sets employed in Section 6 to assess the different methods. Table C.16 presents the characteristics of the standard instances set by set, while Table C.17 give summary statistics of dense and sparse instance sets. Figure C.13 draws a typical precedence structure obtained by merging two RCPSP instances from PSPLIB[5] together.

Set	$\lvert\mathcal{T}\rvert$	$\lvert\mathcal{J}\rvert$				$\lvert\mathcal{J}\rvert/\lvert\mathcal{T}\rvert$	Horizon h			
		mean	min	max	std	mean	mean	min	max	std
1	32	6.55	3	10	1.67	20.47	836.30	486	1117	150.19
2	64	11.60	8	15	1.64	18.12	963.95	740	1288	153.68
3	96	17.70	12	22	2.58	18.44	1297.70	928	1543	166.88
4	128	23.05	17	31	3.05	18.01	1672.35	1272	2055	225.03
5	160	29.70	23	35	3.66	18.56	1953.05	1601	2291	187.93
6	192	35.10	26	53	5.42	18.28	2306.45	1624	3228	397.50
7	224	40.65	35	47	3.54	18.15	2723.45	1999	3539	349.78
8	256	44.80	38	50	3.24	17.50	3039.45	2110	3757	411.91
9	288	51.40	42	60	5.48	17.85	3423.10	2670	3997	401.12
10	320	56.35	46	68	6.43	17.61	3816.95	2924	4517	509.27
11	352	59.25	52	69	3.70	16.83	4056.00	3445	4607	329.44
12	384	66.60	58	79	6.17	17.34	4406.70	3469	5710	531.31
13	416	71.95	61	85	5.95	17.30	4763.65	4180	6307	493.58
14	448	79.25	73	87	4.56	17.69	5202.85	4488	6066	389.12
15	480	85.75	74	97	7.37	17.86	5527.15	4513	6207	519.27

Table C.16: Characteristics of standard instance sets. For each set, we present the number of tasks ($|\mathcal{T}|$), the number of energy-intensive tasks ($|\mathcal{J}|$), the ratio of energy-intensive tasks over the overall number of tasks (in %), and the time horizon. All instances of a set share the same number of tasks (std = standard deviation).

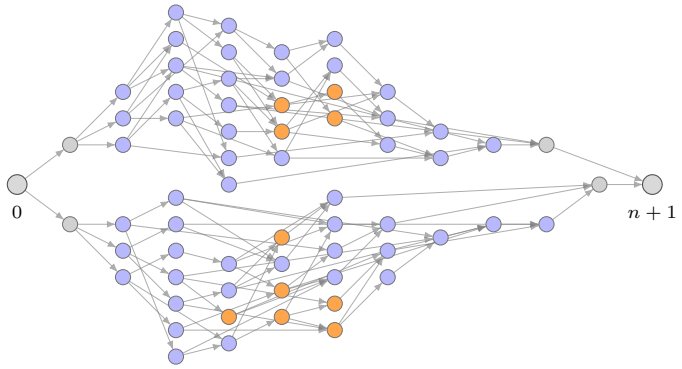


Figure C.13: Precedence structure in the example instance created from two RCPSP instances. In grey: original (from the RCPSP instances) and new global dummy vertices; in orange: energy-intensive tasks.

[5]Available at: https://www.om-db.wi.tum.de/psplib/

	$\lvert\mathcal{T}\rvert$	$\lvert\mathcal{J}\rvert$	$\lvert\mathcal{J}\rvert/\lvert\mathcal{T}\rvert$	Horizon h
mean	128.00	65.00	51.16	4315.06
std	69.12	34.56	0.95	2313.90
min	32.00	17.00	50.45	1098.25
25%	80.00	41.00	50.57	2705.88
50%	128.00	65.00	50.78	4277.05
75%	176.00	89.00	51.30	5978.52
max	224.00	113.00	53.12	7461.30

(a) Summary statistics of dense instances.

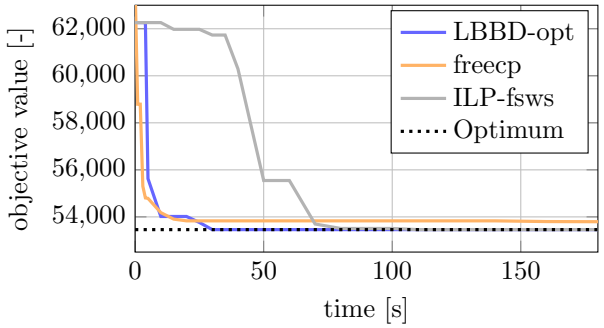
	$\lvert\mathcal{T}\rvert$	$\lvert\mathcal{J}\rvert$	$\lvert\mathcal{J}\rvert/\lvert\mathcal{T}\rvert$	Horizon h
mean	336.00	11.50	3.69	2407.87
std	189.31	5.92	0.70	1114.68
min	32.00	2.00	3.28	764.20
25%	184.00	6.75	3.33	1549.23
50%	336.00	11.50	3.42	2349.10
75%	488.00	16.25	3.67	3360.75
max	640.00	21.00	6.25	4065.30

(b) Summary statistics of sparse instances.

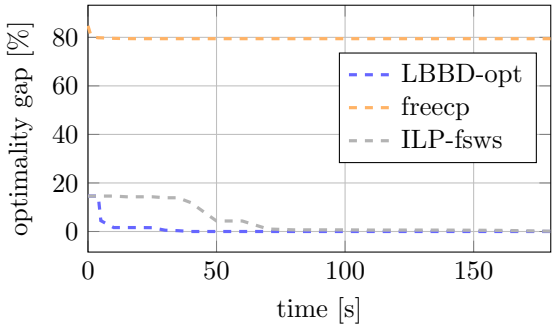
Table C.17: Summary statistics of sparse and dense instances. $|\mathcal{J}|$ and $|\mathcal{T}|$ denote the number of energy-intensive tasks and overall tasks, respectively. The ratio $|\mathcal{J}|/|\mathcal{T}|$ is reported as a percentage. Horizon length and $|\mathcal{J}|$ are averaged across instances within each set.

Appendix D. Evolution of Upper & Lower bounds between approaches

In Figure D.14, the evolution of the value of the upper bound (UB) and of the gap to the best proven lower bound (LB) of methods `LBBD-opt` (without `freecp` warmstart), `ILP-fsws`, and `freecp`, for instance standard_5_5, is presented. This particular instance was selected due to its representativeness of the algorithms' behavior. We can see that the `freecp` is the method having the steepest UB curve (see Figure D.14a). The `LBBD-opt` method follows closely behind, exhibiting a similarly sharp incline. This means that these approaches quickly converge to a good solution. In contrast, `ILP-fsws` takes more time to improve its UB. However, it ends up by finding and proving the optimal solution, as the `LBBD-opt` does, whereas the `freecp` gets stuck with a good but non-optimal solution, which seems unable to improve this solution. Indeed, its LB is very weak compared to the `ILP` approaches (Figure D.14b). For information, in this instance, the `LBBD-opt` generates 18 feasibility cuts and 15 optimality cuts, with an average size of $\mathcal{I}nf$ of 1.333 variables. Moreover, for an overall computation time of 35 seconds, the solver passes around 0.6 seconds calculating the subproblems (considering that 33 subproblems were computed in total, this represents approximately 18 milliseconds per subproblem).



(a) Evolution of the upper bound (UB).



(b) Evolution of the gap of the best proven lower bound (LB).

Figure D.14: Evolution of the UB and the optimality gap during the solution in standard instance 5_5 and $\alpha = 1$.

Appendix E. Boxplots of real improvements over `ILP-fsws`

This section displays the real improvements over `ILP-fsws` in the form of boxplots per set and per α values. Three methods are considered, `LBBD-opt` (`LBBD-heur` in sparse instances), `ILP-cpws` and `freecp`. The real improvements are computed for any method μ as:

$$\delta_\mu = \frac{obj_{\texttt{ILP-fsws}} - obj_\mu}{|obj_{\texttt{ILP-fsws}}|} \cdot 100 \tag{E.1}$$

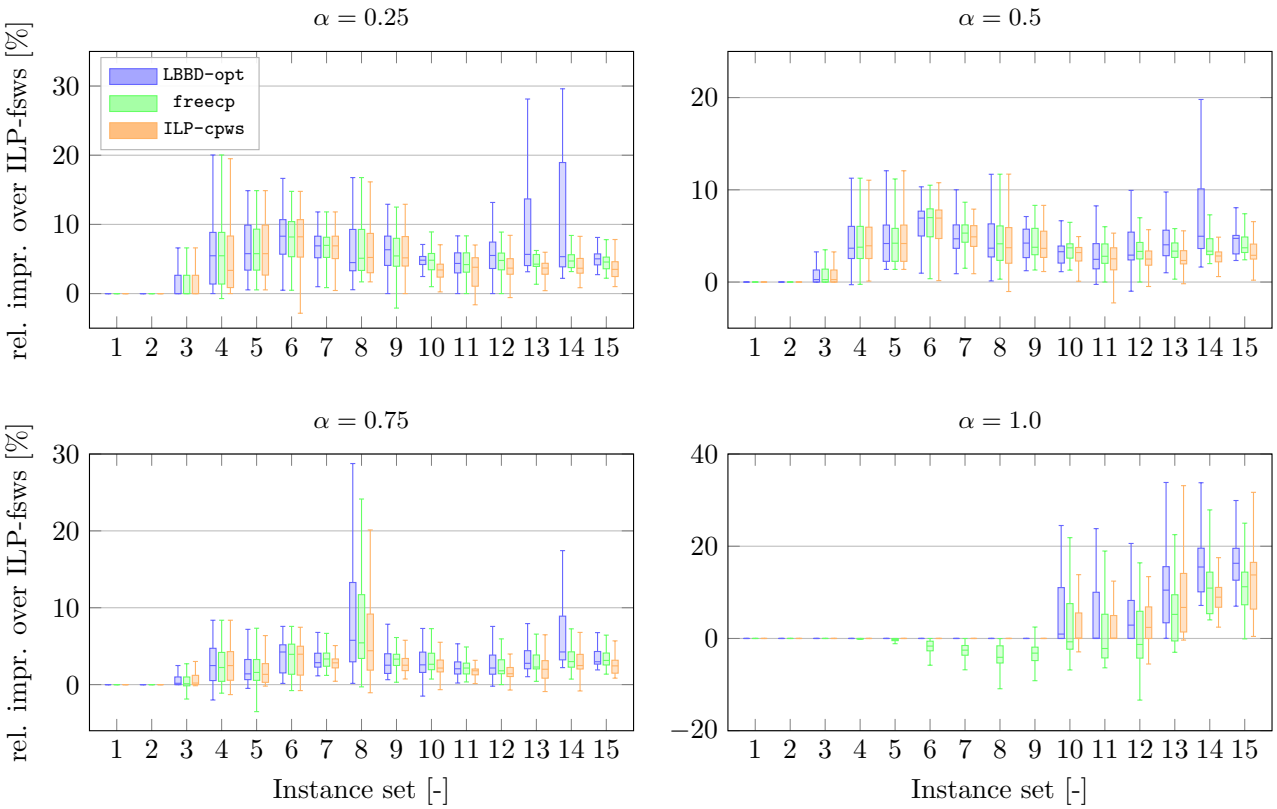


Figure E.15: Real improvements of different methods relative to `ILP-fsws` in standard sets.

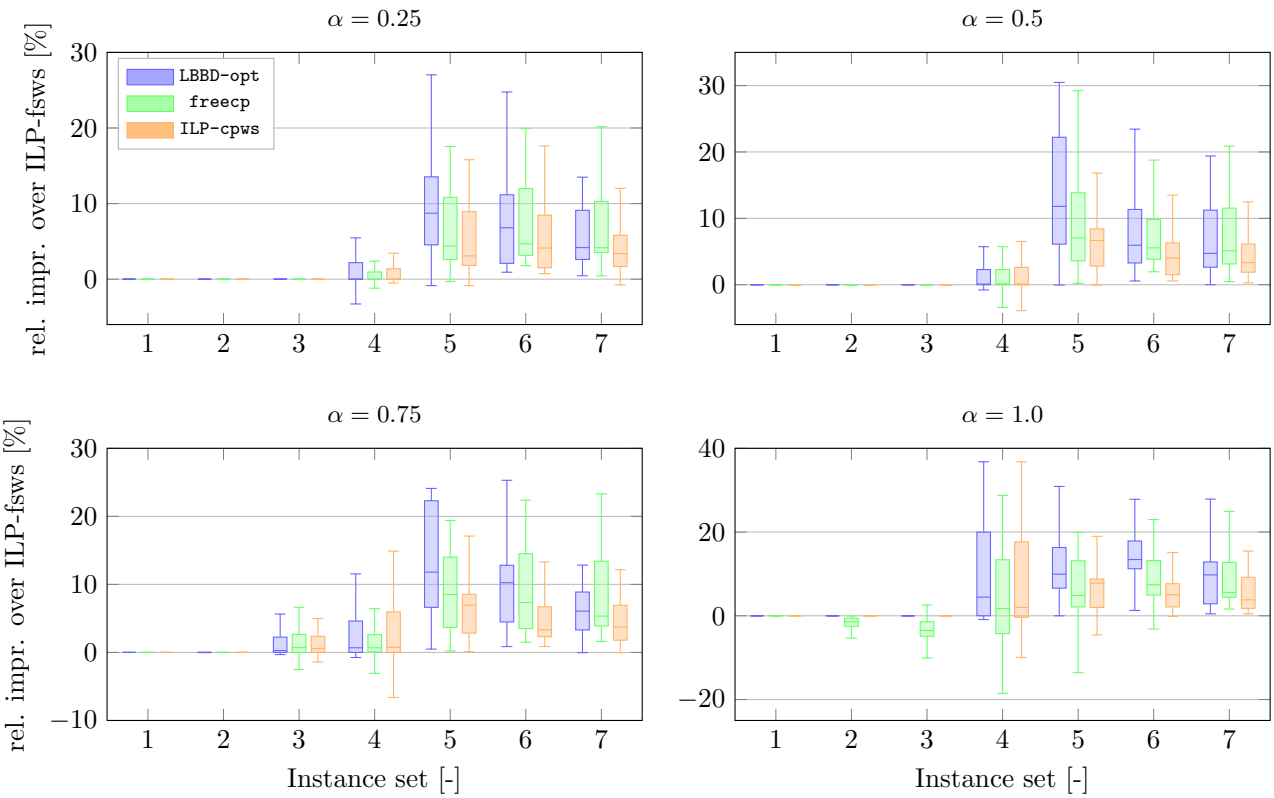


Figure E.16: Real improvements of different methods relative to `ILP-fsws` in dense sets.

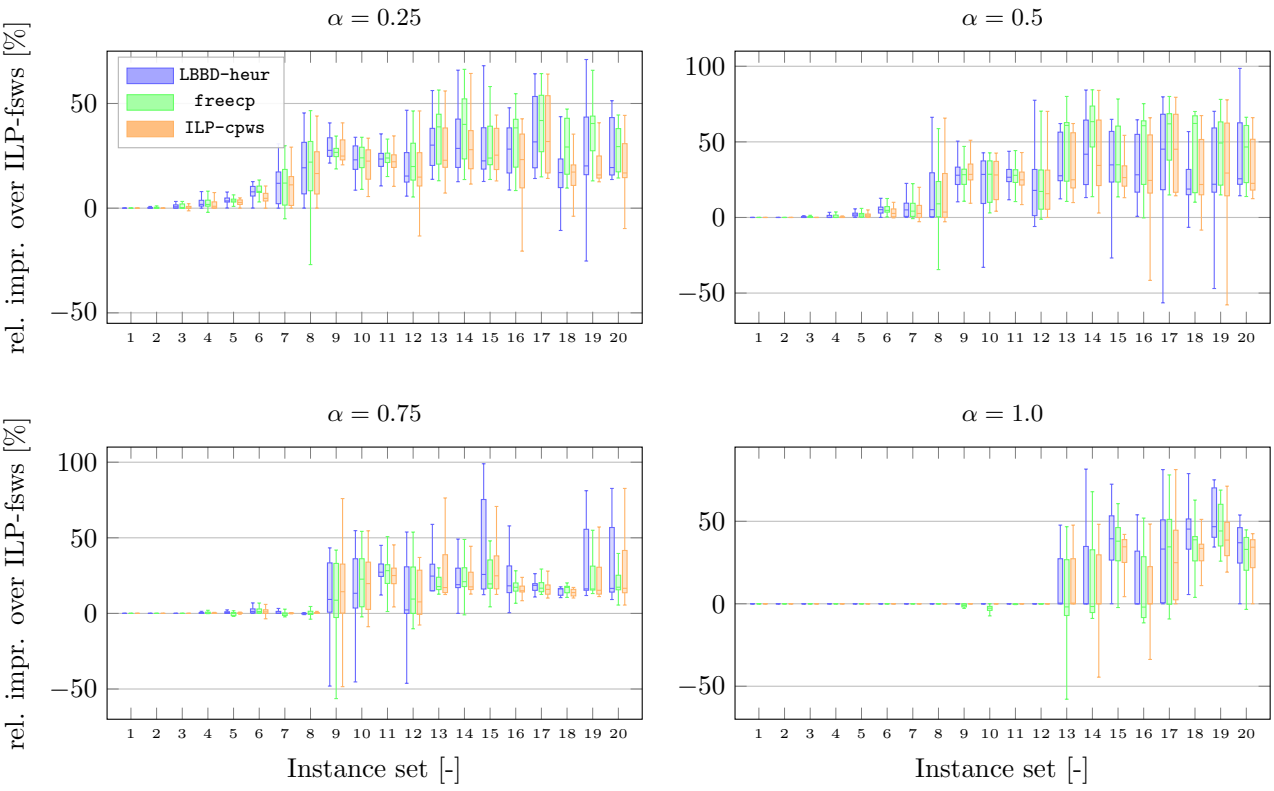


Figure E.17: Real improvements of different methods relative to `ILP-fsws` in sparse sets.

Appendix F. Detailed results for different density sets

This section presents for each density the average of results aggregated per set and per α value. The best results are reported in bold. The computation times are given in seconds, and the gaps in percentages.

		Objective value [-]				Computation Time [s]				Gap to LB [%]			
α	Set	freecp	ILP-cpws	ILP-fsws	LBBD-opt	freecp	ILP-cpws	ILP-fsws	LBBD-opt	freecp	ILP-cpws	ILP-fsws	LBBD-opt
0.25	1	**1.05860**	**1.05860**	**1.05860**	**1.05860**	1321.14	60.03	**11.31**	57.41	10.93	**0.00**	**0.00**	**0.00**
	2	**1.05505**	**1.05505**	**1.05505**	**1.05505**	TLR	225.50	232.72	**186.48**	26.75	**0.25**	0.26	0.26
	3	**1.04645**	**1.04645**	**1.04645**	**1.04645**	TLR	**548.16**	609.66	630.62	38.21	**0.07**	0.10	0.28
	4	**7.32443**	7.41852	7.80674	7.68027	TLR	1142.44	1358.85	**1070.25**	66.95	**1.13**	2.40	1.70
	5	80.93551	82.40018	86.34838	**77.83109**	**TLR**	TLR	TLR	TLR	89.64	24.93	29.50	**3.89**
	6	97.97512	100.03595	105.51415	**97.43803**	**TLR**	TLR	TLR	TLR	172.56	5544.87	5007.17	**41.96**
	7	**82.73975**	84.94907	88.58123	84.16912	TLR	1415.57	1352.68	**1132.79**	72.30	160.65	148.05	**44.96**
0.5	1	**1.11721**	**1.11721**	**1.11721**	**1.11721**	1538.67	136.00	**73.27**	185.64	33.28	**0.00**	**0.00**	0.17
	2	**1.08590**	**1.08590**	**1.08590**	**1.08590**	TLR	**505.29**	644.50	800.25	39.04	**0.46**	0.61	1.13
	3	**1.08238**	**1.08238**	**1.08238**	**1.08238**	TLR	1308.08	1268.98	**1201.41**	71.10	2.19	**1.88**	2.11
	4	16.59966	**16.13410**	18.41283	16.91629	TLR	**1642.80**	1742.26	1661.19	123.92	**2.71**	6.07	3.61
	5	152.09141	161.23074	170.34044	**150.40514**	TLR	TLR	TLR	**1780.22**	87.73	20.75	26.10	**2.21**
	6	194.42347	200.26270	210.00732	**191.02657**	**TLR**	TLR	TLR	TLR	174.98	8111.50	7567.59	**75.73**
	7	**163.94154**	168.47701	176.14825	166.26513	TLR	1413.53	1354.23	**1132.26**	72.69	153.16	148.50	**44.37**
0.75	1	**1.10964**	**1.10964**	**1.10964**	**1.10964**	TLR	382.29	317.62	**264.16**	40.76	0.19	0.12	**0.00**
	2	**1.09154**	1.09175	1.09179	1.09191	TLR	**1474.39**	1561.03	1540.80	65.68	**2.07**	2.73	2.86
	3	10.39699	**10.17224**	10.18724	10.17340	TLR	TLR	TLR	**1799.39**	86.40	6.93	8.03	**6.67**
	4	**28.32943**	29.35689	31.93691	28.91132	**TLR**	TLR	TLR	TLR	165.17	6.73	9.52	**6.53**
	5	234.38472	239.65550	255.20947	**225.22221**	TLR	TLR	TLR	**1757.85**	88.36	50.12	30.37	**2.67**
	6	290.29475	300.76084	314.50048	**281.58823**	**TLR**	TLR	TLR	TLR	178.78	5634.02	2792.82	**41.29**
	7	**242.25874**	252.53526	263.71497	248.55957	TLR	1503.01	1350.43	**1132.21**	72.70	166.58	141.40	**47.32**
1.0	1	**49408.81750**	**49408.81750**	**49408.81750**	**49408.81750**	TLR	292.15	**225.00**	228.09	64.93	0.07	**0.03**	0.16
	2	108772.76421	**107160.87684**	107161.49211	107161.61316	TLR	354.45	282.75	**156.03**	118.42	**0.00**	**0.00**	0.01
	3	144168.30895	139158.31105	139449.32421	**139158.01211**	TLR	755.11	910.36	**436.88**	124.05	0.02	0.27	**0.01**
	4	173414.08050	166458.56800	180866.29849	**163275.10700**	TLR	1616.52	1648.86	**1202.86**	196.45	1.98	10.19	**0.06**
	5	265937.62500	262395.06300	283524.59650	**250761.59450**	TLR	TLR	TLR	**1442.99**	88.88	5.54	12.05	**0.32**
	6	393629.11450	407528.09250	426675.22200	**368332.18250**	TLR	TLR	TLR	**1608.22**	174.00	49.73	15.43	**0.13**
	7	**377964.79000**	391617.81950	411972.01100	379780.83050	TLR	1509.29	1355.02	**1181.05**	72.86	159.47	130.97	**45.95**

Table F.18: Detailed results of the dense density sets. Values are averaged over instances within each set. "TLR" = Time Limit Reached (≥ 1800 s). Best values are reported in bold.

α	Set	Objective value [-] freecp	ILP-cpws	ILP-fsws	LBBD-opt	Computation Time [s] freecp	ILP-cpws	ILP-fsws	LBBD-opt	Final gap to LB [%] freecp	ILP-cpws	ILP-fsws	LBBD-opt
	1	**1.08459**	**1.08459**	**1.08459**	**1.08459**	349.96	48.08	**20.12**	36.81	23.88	**0.00**	**0.00**	**0.00**
	2	**1.05195**	**1.05195**	**1.05195**	**1.05195**	871.22	83.85	252.36	**57.51**	3.14	**0.00**	**0.00**	**0.00**
	3	1.07074	1.07055	1.10652	**1.07026**	1764.58	**340.66**	1564.50	426.34	23.86	**0.25**	12.47	0.26
	4	1.05507	1.06199	1.13507	**1.05270**	TLR	854.79	1796.06	**278.73**	10.84	0.99	23.99	**0.16**
	5	1.04661	1.04776	1.12266	**1.04534**	TLR	816.89	TLR	**446.09**	16.29	0.48	28.09	**0.11**
	6	1.04790	1.05408	1.14501	**1.04505**	TLR	1218.21	TLR	**514.52**	32.56	1.31	31.34	**0.15**
	7	1.05982	1.06266	1.13424	**1.05974**	TLR	990.98	TLR	**504.06**	41.58	0.59	32.16	**0.58**
0.25	8	**1.07104**	1.07438	1.15224	1.07317	TLR	1219.45	TLR	**1201.63**	36.90	**0.54**	33.31	1.46
	9	1.04976	1.03908	1.10634	**1.03680**	TLR	949.61	TLR	**850.48**	18.79	**0.38**	33.59	0.69
	10	**1.01389**	1.02791	1.06565	1.01404	TLR	TLR	TLR	**1296.81**	33.71	1.93	37.79	**0.55**
	11	**1.04572**	1.05748	1.09257	1.04614	TLR	1630.08	1744.89	**1500.07**	25.49	1.77	5.59	**0.67**
	12	**1.03324**	1.04158	1.08769	1.03338	TLR	1740.37	1752.09	**981.44**	27.06	6.73	11.32	**0.51**
	13	3.40062	3.41969	3.60836	**3.31770**	TLR	1783.06	1759.36	**1202.39**	30.88	7.52	16.54	**0.75**
	14	11.19105	11.24702	11.96225	**10.66233**	TLR	TLR	1764.39	**1422.41**	27.40	26.46	30.31	**0.45**
	15	5.21018	5.21865	5.63304	**4.85424**	TLR	TLR	1771.08	**787.89**	17.63	105.91	105.90	**0.06**
	1	**1.14210**	**1.14210**	**1.14210**	**1.14210**	681.41	69.47	**61.51**	72.08	45.16	**0.00**	**0.00**	**0.00**
	2	**1.10034**	**1.10034**	**1.10034**	**1.10034**	1432.25	84.68	294.92	**70.68**	10.12	**0.00**	**0.00**	**0.00**
	3	**1.13696**	1.13726	1.14840	1.13768	TLR	**716.94**	1665.01	824.88	53.42	**0.64**	10.40	1.78
	4	**1.10261**	1.10597	1.16521	1.10494	TLR	**1223.63**	TLR	1391.25	22.87	2.60	18.42	**2.45**
	5	**1.09023**	1.09062	1.14469	1.09044	TLR	1482.18	TLR	**1131.51**	36.78	2.02	19.94	**1.59**
	6	**1.08917**	1.09325	1.16505	1.09037	TLR	1600.48	TLR	**1324.10**	68.95	2.21	22.31	**1.28**
	7	**1.11429**	1.11906	1.17585	1.11633	TLR	**979.56**	TLR	1208.35	78.68	**1.34**	23.72	3.46
0.5	8	**1.14059**	1.15011	1.20299	1.14200	TLR	**1543.41**	TLR	1598.82	71.60	**4.90**	25.19	5.28
	9	**1.09764**	1.10026	1.14592	1.09779	TLR	**1332.94**	TLR	1786.07	35.62	**2.30**	23.68	3.06
	10	**1.02097**	1.03175	1.06224	1.02704	TLR	1761.59	TLR	**1606.72**	68.47	**1.83**	25.99	2.96
	11	**1.08772**	1.10206	1.12132	1.08818	TLR	**1666.37**	1719.87	1794.28	48.82	**2.13**	4.94	2.62
	12	1.06384	1.07465	1.10225	**1.06347**	TLR	**1696.71**	1749.30	1709.05	50.53	8.65	11.94	**2.59**
	13	5.55047	5.49835	5.85881	**5.32971**	TLR	TLR	1758.87	**1584.83**	52.64	9.29	12.46	**2.78**
	14	20.03995	20.39942	21.62985	**19.06893**	TLR	TLR	1763.11	**1605.99**	40.33	10.37	19.50	**1.77**
	15	10.10940	10.35888	11.25391	**9.53414**	TLR	TLR	1769.46	**1418.19**	38.32	167.70	156.43	**0.51**
	1	**1.13832**	**1.13832**	**1.13832**	**1.13832**	901.69	87.81	89.64	**60.90**	75.51	**0.00**	**0.00**	**0.00**
	2	**1.11362**	**1.11362**	**1.11362**	**1.11362**	1746.07	161.22	417.25	**108.25**	21.68	**0.00**	0.16	**0.00**
	3	1.15451	**1.15079**	1.16005	1.15177	TLR	1437.21	1771.71	**1326.31**	88.32	**3.20**	9.50	3.49
	4	1.11382	1.11410	1.14391	**1.11301**	TLR	TLR	TLR	**1722.50**	38.47	4.95	11.00	**3.18**
	5	1.11919	1.11853	1.14569	**1.11369**	TLR	TLR	TLR	**1762.97**	60.59	5.20	12.47	**4.16**
	6	1.12251	1.12756	1.16635	**1.12227**	TLR	TLR	TLR	**1560.15**	107.06	6.43	14.53	**5.06**
	7	**1.12241**	1.13846	1.17222	1.12290	TLR	**1601.52**	TLR	TLR	100.20	6.36	16.27	**5.51**
0.75	8	**1.03726**	1.14741	1.23506	1.10494	TLR	**1716.94**	TLR	1783.66	88.27	9.23	19.98	**7.28**
	9	1.08480	1.08635	1.12071	**1.07815**	TLR	**1677.04**	TLR	TLR	50.83	**5.25**	17.60	6.17
	10	**1.01578**	1.02162	1.05158	1.01884	TLR	TLR	TLR	**1684.74**	98.55	**4.81**	18.69	5.43
	11	1.12130	1.12681	1.14778	**1.11638**	TLR	TLR	**1750.61**	TLR	70.30	6.20	8.44	**5.97**
	12	1.06878	1.07141	1.09077	**1.06523**	TLR	1774.10	**1755.41**	TLR	43.93	6.03	8.51	**5.03**
	13	8.06592	8.05083	8.66011	**7.78350**	TLR	TLR	1758.96	**1666.11**	76.15	7.47	10.99	**5.73**
	14	29.31666	30.21150	31.93878	**28.13772**	TLR	TLR	1763.84	**1759.66**	61.01	18.93	27.20	**4.20**
	15	13.88790	14.20358	15.55150	**13.10518**	TLR	TLR	1769.70	**1742.26**	71.66	26.92	814.91	**1.83**
	1	**18836.80250**	**18836.80250**	**18836.80250**	**18836.80250**	891.27	56.27	**5.98**	52.22	107.84	**0.00**	**0.00**	**0.00**
	2	**36342.00267**	**36342.00267**	**36342.00267**	**36342.00267**	TLR	66.55	**13.09**	62.09	35.22	**0.00**	**0.00**	**0.00**
	3	**45363.12875**	**45363.12875**	**45363.12875**	**45363.12875**	TLR	89.34	72.59	**67.20**	192.71	**0.00**	**0.00**	**0.00**
	4	62794.54909	**62751.56455**	**62751.56455**	**62751.56455**	TLR	145.17	129.38	**80.06**	49.63	**0.00**	**0.00**	**0.00**
	5	76688.37182	**76266.03000**	**76266.03000**	**76266.03000**	TLR	231.32	239.58	**98.22**	96.05	**0.00**	**0.00**	**0.00**
	6	112013.69842	**109582.86315**	**109582.86315**	**109582.86315**	TLR	487.40	520.72	**127.83**	143.99	**0.00**	**0.00**	**0.00**
	7	118167.45105	**114656.45232**	114672.92411	**114656.45232**	TLR	842.11	906.26	**165.40**	120.52	**0.00**	0.03	**0.00**
1.0	8	129669.98737	124555.63158	124856.84789	**124534.03315**	TLR	1199.41	1219.21	**224.64**	137.12	0.03	0.19	**0.00**
	9	183904.69526	177028.07632	179707.40632	**177028.01519**	TLR	1299.15	1382.82	**333.15**	61.48	0.02	0.81	**0.00**
	10	189091.72050	186672.55000	194835.94949	**181718.76700**	TLR	1688.72	1678.52	**288.47**	120.53	2.17	5.83	**0.00**
	11	184661.06550	181347.22400	188558.34750	**177171.48050**	TLR	1597.09	1646.59	**243.89**	90.58	1.50	5.34	**0.01**
	12	221676.40250	215938.58150	228079.62244	**211589.56950**	TLR	1765.82	1749.87	**282.67**	101.02	1.58	6.59	**0.00**
	13	204602.23550	196880.24150	216430.08050	**192963.33359**	TLR	TLR	1758.98	**712.21**	98.62	2.08	11.76	**0.01**
	14	251520.22900	249023.57500	281722.66250	**236034.34960**	TLR	TLR	1764.36	**751.54**	157.92	5.69	17.90	**0.01**
	15	256199.55950	249610.25150	289114.31100	**242278.01933**	TLR	TLR	1770.96	**878.84**	238.85	2.93	17.35	**0.01**

Table F.19: Detailed results for standard density sets. Values are averaged over instances within each set. "TLR" = Time Limit Reached (≥ 1800 s). Best values are reported in bold. Computation times are given in seconds, and gaps are reported as percentages.

		Objective value [-]				Computation Time [s]				Gap to LB [%]			
α	Set	freecp	ILP-cpws	ILP-fsws	LBBD-heur	freecp	ILP-cpws	ILP-fsws	LBBD-heur	freecp	ILP-cpws	ILP-fsws	LBBD-heur
0.25	1	**1.09023**	**1.09023**	**1.09023**	**1.09023**	6.81	9.69	**4.08**	7.50	0.01	**0.00**	**0.00**	**0.00**
	2	**1.07280**	1.07512	1.07656	**1.07280**	873.01	531.25	787.55	**70.82**	5.52	0.62	0.84	**0.00**
	3	1.08411	1.08868	1.09532	**1.08197**	1312.74	1204.90	1255.79	**238.56**	6.20	1.06	1.61	**0.00**
	4	1.15686	1.16207	1.22160	**1.15340**	1649.63	1656.86	1504.31	**572.74**	12.04	1.42	5.47	**0.00**
	5	**1.16648**	1.17881	1.21112	1.16688	1632.16	1577.41	1644.91	**1269.00**	221.80	2.77	5.60	**1.05**
	6	**1.09679**	1.13246	1.20050	1.09843	TLR	TLR	TLR	**1724.52**	11.40	7.27	12.45	**2.98**
	7	1.16676	1.17364	1.31607	**1.15752**	TLR	**1575.86**	1579.20	1739.10	219.30	3.13	13.85	**1.64**
	8	1.31094	1.30386	1.63713	**1.27765**	TLR	1684.67	1661.99	**1240.86**	1126.71	8.02	26.52	**3.57**
	9	1.04436	1.04715	1.45101	**1.02183**	**TLR**	TLR	TLR	TLR	14.07	6.11	28.25	**3.49**
	10	**1.06016**	1.08312	1.38801	1.06168	TLR	TLR	TLR	**1567.72**	14.77	5.44	26.07	**2.44**
	11	**1.08755**	1.11401	1.49809	1.09618	**TLR**	TLR	TLR	TLR	119.08	4.89	28.55	**3.11**
	12	**1.08884**	1.19911	1.48523	1.14369	TLR	TLR	TLR	**1576.00**	123.47	8.37	26.12	**2.84**
	13	**0.88733**	1.00737	1.41042	0.96489	**TLR**	TLR	TLR	TLR	8.14	7.45	26.72	**3.82**
	14	**0.92505**	1.01825	1.70346	1.00792	TLR	TLR	TLR	**1739.54**	113.36	7.42	32.61	**3.51**
	15	2.20333	2.29312	3.52375	**0.97253**	**TLR**	TLR	TLR	TLR	15.39	7.73	28.23	**5.47**
	16	**0.89375**	1.06637	1.36038	0.97866	**TLR**	TLR	TLR	TLR	181.90	7.23	23.91	**4.63**
	17	**0.80364**	0.86932	1.44146	0.86289	**TLR**	TLR	TLR	TLR	41.83	3.29	29.10	**2.68**
	18	**0.77309**	0.93138	1.13899	0.91351	TLR	TLR	TLR	**1752.57**	5.57	5.87	20.36	**3.42**
	19	2.79964	4.80577	7.19811	**2.63461**	TLR	TLR	TLR	**1757.45**	11.68	11.78	28.85	**4.04**
	20	3.51991	3.74243	5.45435	**2.91696**	TLR	1720.00	**1631.06**	1718.24	80.07	5.79	32.91	**2.61**
0.5	1	**1.08398**	**1.08398**	**1.08398**	**1.08398**	7.36	7.93	**3.30**	5.65	0.01	**0.00**	**0.00**	**0.00**
	2	**1.10739**	**1.10739**	**1.10739**	**1.10739**	687.02	53.92	65.37	**45.40**	5.93	**0.00**	**0.00**	**0.00**
	3	**1.09877**	1.10299	1.10299	1.09924	1231.91	835.48	810.02	**151.30**	8.06	0.78	0.78	**0.00**
	4	**1.17812**	1.18087	1.18684	1.17823	1045.44	673.46	848.34	**169.69**	5.24	0.42	0.99	**0.00**
	5	1.15669	1.16182	1.19310	**1.15150**	1631.32	1535.99	1545.09	**833.38**	434.39	1.73	4.31	**0.33**
	6	1.12451	1.14394	1.18943	**1.11696**	TLR	TLR	TLR	**1371.63**	21.04	5.72	9.75	**1.99**
	7	1.16885	1.18468	1.28593	**1.15879**	1800.00	1367.26	1521.87	**1037.38**	485.69	3.97	10.16	**0.56**
	8	1.31984	1.23629	1.61115	**1.21985**	1757.01	1312.71	1451.92	**987.61**	2120.18	3.48	19.61	**1.50**
	9	1.05489	**1.01059**	1.49786	1.04348	TLR	**1692.87**	TLR	1703.18	25.63	5.23	29.67	**2.15**
	10	**1.06482**	1.07143	1.50049	1.13253	TLR	TLR	TLR	**1458.43**	25.86	5.03	29.46	**2.55**
	11	1.14481	1.16425	1.62094	**1.13989**	TLR	TLR	TLR	**1474.52**	215.05	7.34	31.89	**2.35**
	12	**1.13701**	1.14284	2.01879	1.22036	TLR	1761.86	1779.82	**1657.10**	216.25	6.09	28.39	**3.33**
	13	**0.75005**	0.96838	1.51021	0.93357	TLR	TLR	**TLR**	TLR	14.18	7.63	29.20	**4.59**
	14	**0.77957**	1.03886	2.27411	0.97402	TLR	TLR	TLR	**1726.02**	198.45	12.30	41.80	**5.40**
	15	2.94050	3.41132	5.23333	**2.66294**	TLR	TLR	TLR	**1673.65**	21.27	10.19	32.20	**4.02**
	16	**0.70172**	0.98988	1.41011	0.91431	TLR	TLR	TLR	**1456.95**	358.91	9.29	27.89	**3.46**
	17	**0.58883**	0.71334	1.47722	0.75661	TLR	TLR	TLR	**1787.09**	84.75	5.19	32.00	**3.80**
	18	**0.52446**	0.75656	1.17983	0.79561	TLR	TLR	TLR	**1744.16**	12.31	8.99	27.09	**6.51**
	19	7.64672	7.41469	12.32683	**4.11329**	TLR	TLR	TLR	**1741.21**	15.61	14.61	32.93	**4.84**
	20	5.91176	6.33539	9.24639	**0.81636**	TLR	1695.92	**1692.25**	TLR	148.49	7.39	29.08	**3.65**
0.75	1	**1.06916**	**1.06916**	**1.06916**	**1.06916**	5.64	8.59	**3.17**	6.35	0.01	**0.00**	**0.00**	**0.00**
	2	**1.09688**	**1.09688**	**1.09688**	**1.09688**	150.14	64.14	67.11	**48.15**	0.01	**0.00**	**0.00**	**0.00**
	3	**1.13331**	**1.13331**	**1.13331**	**1.13331**	420.79	61.12	**21.59**	63.57	4.09	**0.00**	**0.00**	**0.00**
	4	**1.11885**	1.12014	1.12545	1.11938	924.66	710.18	664.26	**156.52**	7.65	0.16	0.62	**0.00**
	5	1.13391	1.13058	1.13055	**1.12281**	1632.00	1220.33	1178.31	**472.97**	664.45	1.04	1.02	**0.23**
	6	**1.08118**	1.09257	1.13770	1.08174	TLR	1674.21	1552.25	**1292.29**	34.18	2.40	5.37	**1.00**
	7	1.12975	1.12389	1.13115	**1.11830**	1800.00	1010.98	970.81	**360.32**	881.07	1.48	2.12	**0.16**
	8	1.13502	**1.12639**	1.13347	1.13284	1725.88	922.05	1004.40	**620.79**	616.88	**1.11**	1.82	1.47
	9	1.02646	**1.00199**	1.10871	1.01501	TLR	TLR	1767.38	**755.29**	36.69	8.85	23.06	**1.06**
	10	**1.03711**	1.06779	1.42239	1.11413	TLR	TLR	TLR	**1431.29**	36.39	7.19	25.18	**0.62**
	11	1.12342	1.14591	1.62332	**1.11716**	TLR	TLR	TLR	**1106.89**	293.88	7.55	31.58	**1.45**
	12	**1.07760**	1.13723	1.36671	1.15214	TLR	1721.66	1645.78	**723.05**	377.97	11.50	21.98	**0.26**
	13	0.94024	**0.59146**	1.24210	0.76857	**TLR**	TLR	TLR	TLR	35.66	3.79	27.60	**3.06**
	14	**0.75900**	0.84168	1.27255	0.76369	TLR	TLR	TLR	**1186.79**	650.40	10.84	32.00	**1.59**
	15	16.18649	16.69934	27.73834	**3.00898**	TLR	TLR	TLR	**1365.55**	28.23	7.26	28.84	**2.67**
	16	0.58644	0.73353	0.95812	**0.58406**	TLR	TLR	TLR	**1301.02**	1038.00	13.45	24.76	**2.73**
	17	**0.42141**	0.55061	0.72388	0.54802	TLR	TLR	TLR	**1601.44**	225.64	5.22	23.43	**3.56**
	18	**0.27476**	0.28152	0.32659	0.27863	**TLR**	TLR	TLR	TLR	37.51	5.51	18.54	**4.20**
	19	11.83949	19.55914	30.76335	**8.54565**	TLR	TLR	TLR	**1381.50**	25.20	17.93	35.64	**4.11**
	20	9.08337	10.43514	15.64535	**7.84095**	TLR	1621.19	1565.19	**1357.25**	243.00	8.07	25.48	**2.71**
1.0	1	**3098.32700**	**3098.32700**	**3098.32700**	**3098.32700**	181.35	8.19	**1.17**	7.30	1576.27	**0.00**	**0.00**	**0.00**
	2	**6865.85800**	**6865.85800**	**6865.85800**	**6865.85800**	211.71	23.41	**2.83**	20.50	8952.38	**0.00**	**0.00**	**0.00**
	3	**13816.60667**	**13816.60667**	**13816.60667**	**13816.60667**	295.50	33.55	**2.95**	29.96	3.10	**0.00**	**0.00**	**0.00**
	4	**9934.80000**	**9934.80000**	**9934.80000**	**9934.80000**	702.16	53.06	**7.09**	51.11	10.77	**0.00**	**0.00**	**0.00**
	5	**13695.72000**	**13695.72000**	**13695.72000**	**13695.72000**	1433.40	61.35	**19.29**	56.42	28.88	**0.00**	**0.00**	**0.00**
	6	**14319.59000**	**14319.59000**	**14319.59000**	**14319.59000**	TLR	75.46	**30.99**	64.80	43.56	**0.00**	**0.00**	**0.00**
	7	**22062.46000**	**22062.46000**	**22062.46000**	**22062.46000**	TLR	77.38	**46.03**	62.71	51.29	**0.00**	**0.00**	**0.00**
	8	**8876.83400**	**8876.83400**	**8876.83400**	**8876.83400**	1624.82	77.92	**44.12**	64.31	1289.57	**0.00**	**0.00**	**0.00**
	9	28228.93167	**27995.23500**	**27995.23500**	**27995.23500**	TLR	155.24	275.05	**70.50**	35.64	**0.00**	**0.00**	**0.00**
	10	20909.16000	**20370.97444**	**20370.97444**	**20370.97444**	TLR	140.12	227.69	**67.03**	49.85	**0.00**	**0.00**	**0.00**
	11	21746.32750	**21695.36000**	**21695.36000**	**21695.36000**	TLR	366.63	937.43	**71.25**	1105.64	**0.00**	0.06	0.01
	12	33952.92500	**33949.77000**	**33949.77000**	**33949.77000**	1799.98	123.14	323.16	**66.11**	1450.54	**0.00**	**0.00**	**0.00**
	13	44945.87100	43098.60500	48287.74300	**43098.60435**	TLR	589.82	1476.29	**86.89**	52.11	**0.00**	13.54	**0.00**
	14	31407.00200	32898.55400	40393.43799	**29029.53100**	TLR	1169.30	1332.30	**93.02**	1511.01	15.24	19.30	**0.00**
	15	42279.75200	45475.15200	64708.47900	**40527.99300**	TLR	1247.36	1766.16	**99.62**	311.96	14.26	37.11	**0.00**
	16	47490.91900	49334.45800	54347.68000	**44503.74900**	TLR	1222.40	1325.24	**92.63**	1805.29	14.29	15.78	**0.00**
	17	36084.89222	36466.57333	54024.20667	**33581.20889**	TLR	1111.14	1727.34	**92.72**	1565.81	8.76	34.57	**0.00**
	18	47058.80700	47496.28100	74651.44400	**40888.69999**	TLR	1452.08	1676.58	**111.32**	1045.68	13.46	44.17	**0.00**
	19	33712.84100	36922.42400	66602.83000	**29024.95700**	TLR	1314.25	TLR	**102.88**	66.78	18.97	53.55	**0.00**
	20	52712.91300	53078.60900	76349.92300	**48569.68783**	TLR	1520.83	1548.72	**131.62**	327.90	7.06	32.22	**0.00**

Table F.20: Detailed results of the sparse density sets. Values are averaged over instances within each set. "TLR" = Time Limit Reached (≥ 1800 s). Best values are reported in bold.

Appendix G. Monolithic MILP formulation for Flexible Job Shop with Machine States and Time-Of-Use

The following monolithic MILP models the Flexible Job Shop with Machine states and TOU problem. The binary decision variable $x_{jt}^k = 1$ only if an operation $j \in \mathcal{T}$ starts on machine $k \in \mathcal{M}_j$ at time $t \in \mathcal{I}$. Binary decision variables $z_{lm} = 1$ only if the machine is performing an optimal SPACES transition between the intervals l and m. Let $C_{\max}$ be a continuous decision variable. We denote as p_j^k the processing time of task $j \in \mathcal{T}$ on machine k, and k_0 the energy-intensive machine.

$$\min \; C_{\max} \tag{G.1}$$

$$s.t. \quad \sum_{t\in\mathcal{I}} \sum_{k\in\mathcal{M}_j} x_{jt}^k = 1 \qquad \forall j \in \mathcal{T} \tag{G.2}$$

$$\sum_{t\in\mathcal{I}} \sum_{k_j\in\mathcal{M}_j} x_{jt}^{k_j} - \sum_{t\in\mathcal{I}} \sum_{k_i\in\mathcal{M}_i} (t + p_i^{k_i}) x_{it}^{k_i} \geq 0 \qquad \forall (i,j) \in \mathcal{T}\times\mathcal{T}, i \prec j \tag{G.3}$$

$$\sum_{j\in\mathcal{T}|k\in\mathcal{M}_j} \sum_{l=\max\{1,t-p_j^k+1\}}^{t} x_{jl}^k \leq 1 \qquad \forall k \in \mathcal{M}, t \in \mathcal{I} \tag{G.4}$$

$$\sum_{j\in\mathcal{J}} \sum_{t\in \text{StartTime}\cup\text{EndTime}_j} x_{jt}^{k_0} = 0 \tag{G.5}$$

$$\sum_{j\in\mathcal{J}} \sum_{t'=\max\{1,t-p_j^{k_0}+1\}}^{t+1} x_{jt'}^{k_0} + \sum_{l=1}^{t+1} \sum_{m=t+1}^{h+1} z_{lm} = 1 \qquad \forall t \in \mathcal{I} \tag{G.6}$$

$$\sum_{t\in\mathcal{I}} \sum_{k\in\mathcal{M}_j} (t + p_j^k) x_{jt}^k \leq C_{\max} \qquad \forall j \in \mathcal{T} \tag{G.7}$$

Constraints (G.2) ensure all operations are scheduled. Constraints (G.3) enforce the precedence constraints, while constraints (G.4) prevent operations from overlapping on machines. Constraints (G.5) and (G.6) model the machine state part of the problem. Finally, constraints (G.7) monitor the makespan (G.1) of the problem.